COMSATS University Islamabad, Vehari Campus, Pakistan

Department of Computer Science

Online Food Ordering and Delivery Website

Final Year Project Documentation
for the Degree of

Bachelor of Science in Computer Science

Submitted By

Muhammad Salman FA22-BCS-034
Muhammad Rehan FA22-BCS-043

Supervisor

Qaizar Javed

Session: FALL 2022-2026

Dedication

I dedicate this work to my parents, whose unwavering support, sacrifices, and prayers have been the foundation of my success. Their encouragement, patience, and belief in my abilities have guided me throughout this journey.

I also dedicate this to my teachers and mentors who inspired me to pursue knowledge with dedication and integrity.

Acknowledgement

All praise and gratitude are due for the successful completion of this project. The authors would like to express their sincere appreciation to their supervisor for continuous guidance, valuable suggestions, and constructive feedback throughout the development of this thesis. The supervision and mentorship provided significant support in completing this work successfully.

Special thanks are extended to the faculty members of the Department of Computer Science for their academic support and encouragement during the degree program.

The authors are also deeply grateful to their parents and families for their unwavering support, motivation, and prayers, which have been a constant source of strength.

Finally, appreciation is extended to everyone who directly or indirectly contributed to the successful completion of this project.

Certificate of Approval

This is to certify that the Final Year Project titled **Online Food Ordering and Delivery Website** has been developed by **Muhammad Salman (FA22-BCS-034)** and **Muhammad Rehan (FA22-BCS-043)** under the supervision of **Qaizar Javed** and that in his opinion, the project is adequate in scope and quality for the award of the degree of Bachelor of Science in Computer Science..

_____

Supervisor

_____

External Examiner

_____

Head of Department
Department of Computer Science

Plagiarism and AI Content Certificate

Date: _____

It is hereby certified that the similarity index (as generated by the Turnitin Originality Report) of the [**Report Type: SRS / SDD / Thesis / Final Year Project Report**] submitted by:

- **Muhammad Salman (FA22-BCS-034)**
- **Muhammad Rehan (FA22-BCS-043)**

titled:

"[Online Food Ordering and Delivery Website]"

is [**XX%**] which is within the permissible limits as defined by the Higher Education Commission (HEC) rules and regulations. The similarity index report is attached herewith.

Furthermore, the AI-generated content detection index (if applicable as per institutional policy) is reported as [**XX%**]. The report has been reviewed, and the content is found to comply with university guidelines regarding ethical use of Artificial Intelligence tools.

The report is recommended for submission.

Supervisor Name: _____

Designation: _____

Signature: _____

Department: Department of Computer Science

University: COMSATS University Islamabad, Vehari Campus

Executive Summary

The Online Food Ordering and Delivery Platform is designed to rev olutionize the way consumers access food services while enhancing operational efficiency for restaurant owners. This platform enables users to browse a variety of restaurants and their menus, place orders seamlessly, and track deliveries in real-time. Key features include user authentication, multiple payment options, and a responsive de sign for accessibility across devices. The implementation of a chatbot for customer support further enhances user experience by providing immediate assistance. Through rigorous testing, the system demon strates reliability, security, and ease of use, fulfilling both functional and non-functional requirements. This project not only meets the current demands of the food delivery market but is also equipped to adapt to future trends, making it a valuable tool for both consumers and food service providers.

Abbreviations

Abbreviation	Full Form
SRS	Software Requirements Specification
SDD	Software Design Document
API	Application Programming Interface
UI	User Interface
DB	Database

Table of Contents

List of Tables

List of Figures

Chapter 1
Introduction

1.1 Introduction

The software in question is an online system used for ordering food from various restaurants. This application will help users to browse through various restaurant menus and order foods conveniently. In addition, payment and food delivery tracking are among other functions of the application. This software application has the ability to assist restaurant owners to register and manage their accounts. As it places much emphasis on user experience, the software will help the user make the ordering procedure easy by providing a responsive design that is available for all types of mobile devices. Various functionalities such as registration and management of user account, food ordering and payment, restaurant management, food delivery, rating and review, responsive design, and chatbot have been included in this software.

1.2 Vision Statement

The online food ordering and delivery system will be able to cater to various parties, which include the customers, the restaurant owners, the delivery agents, and the system managers. The fast-paced world we live in has posed certain problems for consumers, who have to deal with issues such as not having easy access to the menus of different restaurants, difficulties in ordering food, late deliveries, and no means to track their orders.

The primary purpose of the online food ordering and delivery system is to offer a one-stop platform for users where they can easily place an order and track it until it reaches them.

For the long run, the idea is to come up with a sustainable, efficient, and effective platform that can bring about more satisfaction among customers and also help the restaurant simplify its process as well as the delivery process.

In contrast to conventional solutions, this one puts stress on fast reaction, automation, and a great customer experience. Also, it will be developed in such a way that allows for future integration of new technologies, such as developing mobile applications and artificial intelligence.

1.3 Related System Analysis

There are several existing food order and delivery systems, but there are different aspects in which they excel or fail. This analysis will compare popular systems with the proposed one.

The food delivery services offered by many companies include Foodpanda, whose role is to help customers locate their desired restaurant and order food while also being able to track the progress of their deliveries.Even with all those features and high popularity, Foodpanda often suffers from delays in deliveries, poor customer support responses, and expensive service. Also, smaller restaurants may find difficulties with promotion.

The other famous platform is called Uber Eats. It boasts an effective delivery system and a user-friendly interface. The platform also includes real-time tracking and recommendations. Uber

Eats' weaknesses can be identified as high delivery costs compared to the norm, lack of coverage in smaller cities, and relying on third parties.

Thirdly, Zomato is a popular website and app that combines restaurant searching with meal delivery. It provides detailed restaurant reviews and ratings and allows for browsing menu. Although it provides good details about each restaurant, its delivery service may be unreliable and sometimes orders may contain mistakes and delays in updates.

It will be important for the proposed application to integrate those advantages while fixing shortcomings in terms of poor customer support, unfair prices and charging too much money for the service. The proposed application should provide fast and effective communication via an integrated chatbot. Moreover, it should have reasonable prices and minimal service fees.

Another aspect of the application is better handling of deliveries through efficient tracking and delivery person assignment. In addition, any small restaurant should have an opportunity to join this system easily.

The suggested system has some important advantages in comparison to existing systems. It is focused on the convenience of the user and efficient operation of the system in general. For instance, it will use the most recent technologies such as JWT security protocol, responsive design, and instant updates.

1.4 Project Deliverables

A number of deliverables will be generated by this Online Food Ordering and Delivery System, thus proving the success of the whole process of designing, developing and implementing the system. In order to provide the deliverables covering all stages of software development life cycle (planning, design, development, testing, implementation), the following deliverables are required.

Firstly, it should be mentioned about deliverable – the completed web-based application which provides customers with access to menus and order tracking system and allows them to search the desired restaurants and their menus, to place orders and track deliveries. Moreover, there are special modules for restaurant owners and managers to control the menus and orders and for system administrator to control the system operation.

Secondly, there is one more deliverable – the Software Requirements Specification (SRS). This deliverable provides an elaborate system requirements description which includes functional and non-functional requirements along with system boundaries and limitations.

Also, the Software Design Document (SDD) is one more deliverable that will cover system architecture, database design, and detailed module-level design information. The Software Design Document describes the interaction between all parts of the system and will serve as technical specifications for implementation.

Moreover, the project will offer a properly structured database that will contain user information, restaurant details, order information, payments, and feedback from customers.

Additionally, artifacts related to testing, including test cases, test plan, and test report will be delivered. The testing stage will guarantee that the system works properly and performs all required actions.

Moreover, there will be produced deployment outputs, such as installation files, web hosting, and user guides. Web hosting of the system on a remote server allows for testing the developed software by real users.

Table 1.1: Comparative Analysis of Existing Systems

Application Name	**Identified Weaknesses**	**Proposed Improvements in Pizza O' Clock**
Foodpanda	• High delivery and service charges • Delayed deliveries during peak hours • Limited support for small/local restaurants	• Cash on Delivery and Stripe payment options reduce financial barriers • Real-time rider tracking with live order status (Preparing, Rider On Way, Delivered) • Simple restaurant owner onboarding with dedicated Owner role and restaurant profile management
Uber Eats	• High delivery fees • Limited availability in smaller cities • Dependency on third-party drivers	• Wallet system with transparent transaction tracking • Delivery zone configuration allowing restaurants to define their own service areas • Integrated Rider role with ride count tracking and availability management
Zomato	• Inconsistent delivery service • Order inaccuracies • Slow customer support response	• OTP-based delivery verification (rider_otp and user_otp) ensures accurate order handoff • Real-time in-app messaging between rider and user per order • AI-powered chatbot with persistent chat history for instant customer support

Finally, documentation will be provided that will instruct both users and system administrator on how to properly work with the system.

List of Deliverables:

1. **Web-Based Application System** – A fully functional online platform that enables users to order food, restaurants to manage menus, and delivery personnel to handle deliveries efficiently.

2. **Software Requirements Specification (SRS)** – A comprehensive document detailing system requirements, scope, and constraints to guide development.

3. **Software Design Document (SDD)** – A document describing system architecture, database schema, and module-level design.
4. **Database System** – A structured and secure database for managing users, orders, payments, restaurants, and reviews.
5. **Testing Artifacts** – Includes test cases, test plans, and reports to ensure system quality, reliability, and performance.
6. **Deployment and Hosting** – A deployed version of the system on a web server for real-time usage and demonstration.
7. **User Documentation** – User guides and manuals to help customers, restaurant owners, and administrators use the system effectively.
8. **Presentation and Demo** – Final project presentation slides and system demonstration for evaluation purposes.

1.5 System Limitations / Constraints

While the Online Food Ordering and Delivery System may seem like an effective and convenient solution, there are quite a few limitations and constraints that should be considered in the development of the system.

On the technical side, the proposed system requires an active internet connection for all its users, be they clients, restaurant owners, or drivers. In addition, the system utilizes various third-party APIs such as those for payments (e.g., Stripe).

The hardware and software limitations will also be important in determining the system's performance. Specifically, since the application is developed for use in the web browser, it might not be optimized for old hardware or outdated software versions.

Time and financial limitations play a crucial role in the creation of the project. Being a final-year project, the system is created within a time limitation that might limit the use of some advanced functions, for example, artificial intelligence recommendations or complex delivery algorithms. In addition, the system may face some budget limitations while integrating third-party services or choosing hosting services.

Moreover, scalability is one of the aspects that should be considered while creating the system. Although the system is designed to operate with a moderate user amount, it will need further optimization and infrastructure enhancement to ensure its large-scale functioning.

Key Limitations:

- **Internet Dependency** – Requires stable internet connectivity for proper functioning.
- **Third-Party Services** – Dependency on external payment gateways and APIs.
- **Limited Time and Budget** – Constraints on implementing advanced features.
- **Performance Constraints** – May face slowdowns under heavy traffic without scaling.
- **Platform Limitation** – Primarily web-based, limited mobile optimization.

1.6 Tools and Technologies

Table 1.2: Tools and Technologies Used in the Project

Tool / Technology	**Version**	**Purpose**
HTML, CSS, JavaScript	HTML5, CSS3, ES6	Used for designing the structure, styling, and interactivity of the user interface.
Bootstrap	5.x	Provides responsive design and pre-built UI components for better user experience.
Django	18.x	Backend runtime environment for handling server-side logic and requests.
MySQL	6.x	SQL database for storing user data, orders, menus, and reviews.
JWT (JSON Web Token)	Latest	Provides secure authentication and authorization mechanism.
Visual Studio Code	Latest	Code editor used for development with extensions support.
Postman	Latest	API testing tool for verifying backend endpoints.
Git & GitHub	Latest	Version control and code repository management.

1.7 Relevance to Course Modules

The development of the Online Food Ordering and Delivery System reflects the practical application of multiple academic course modules studied throughout the degree program. This project integrates theoretical concepts with real-world implementation, demonstrating a comprehensive understanding of software development principles.

Programming Fundamentals and **Object Oriented Programming (OOP)** have been applied to design the logic and modular architecture of the system. The major tenets of OOP like encapsulation, abstraction, and modularity have been used to develop modules that help process user authentication, order processing, and restaurant management modules.

Concepts learnt in the **Database Systems** course have been used extensively to design and manage the database. Concepts like data modeling, normalization, and relationships have been applied in developing the collection tables in case of users and orders, restaurants and reviews. Proper query handling will help achieve quick retrieval and storing of data.

The **Software Engineering** course has found its application in following structured methodologies such as requirement analysis, system design, implementation and testing in this course. Development of SRS and SDD is an indication of proper documentation practices. Modular architecture of the system is yet another outcome of software engineering learning.

The topics discussed in the **Web Development** class have been extensively used in the creation of the front-end and back-end of the application. Some of these topics include the use

of HTML, CSS, and JavaScript in creating the interfaces and using Node.js and Express.js in creating the back-end servers and API.

On the other hand, concepts discussed in the **Computer Networks** subject are also used in developing networked applications through RESTful APIs and HTTP Requests/Responses for networking and communications among applications.

Furthermore, Information Security concepts have been deployed in the form of authentication through JSON Web Tokens (JWT) to make sure that the information is safe and protected in the process of developing the software application.

Finally, concepts from the **Human-Computer Interaction (HCI)** subject have been effectively used in designing a user-friendly interface ensuring that the user can access the software application and interact with it.

This means that this software application project successfully illustrates how all the concepts have been used to develop the software application.

1.7.1 Programming Fundamentals

Programming Fundamentals was helpful in determining the logical steps that had to be undertaken in designing the application. Conditional statements, looping structures, and function calls were used in the process of developing the functionalities of the program, including user login, order placement, and menu management. Object-oriented programming (OOP) design principles were considered in the development of the source code, and this included the principles of modularity and encapsulation. Each part of the program operates on its own without affecting other parts.

1.7.2 Data Structures and Algorithms

The use of Data Structures and Algorithms made it possible for the effective management of data in the system. Arrays and Objects were employed for storing and processing data associated with the user, menu item, and order. Searches were made possible through algorithms, trying to make selections of the restaurant and menu item based on the user's preference. Sorting algorithm was employed in the sorting of the restaurants based on their popularity or ranking.

1.7.3 Database Management Systems

Database Management System design concepts were utilized in the design process of the database effectively. The data was partitioned into groups within the database through MongoDB, for instance, users, restaurants, orders, and reviews. Data modeling approaches were applied to link the various data entities together. Also, normalization principles were applied to avoid any duplication of data in the database. SQL queries were generated for inserting, updating, and deleting records from the database.

1.7.4 Software Engineering

The principles of Software Engineering were applied during the whole process of developing the system.The system has been developed using a systematic approach based on the software development life cycle (SDLC), which includes requirement analysis, designing, developing, testing,

and deploying the system.Diagrams like use-case diagrams and data flow diagrams using Unified Modeling Language (UML) have been developed for visualization of the system. The preparation of the SRS and SDD has helped in effective requirement and design communication. Different types of testing techniques have been adopted for system verification and validation.

1.7.5 Other Relevant Courses

The subjects that helped me develop the system included other subjects such as Web Development, Computer Networking, Cyber Security, and HCI (Human-Computer Interaction). The concepts from Web Development such as HTML, CSS, JavaScript, and Bootstrap were instrumental in developing the interface of the application. The concepts of Computer Networking played a major role in comprehending communication between client and server and also API implementation. The concepts of Cyber Security were used by way of authentication via JWT. Concepts of HCI were also utilized for designing the application interface.

1.8 Chapter Summary

This chapter has introduced the background and setting of the proposed system, especially with regard to existing problems that have been encountered, together with what is required. The vision of the proposed system was then stated to provide an indication of the intended effects that would be produced. Comparison with other existing systems revealed the deficiencies in these systems. Project deliverables, system constraints, technology choices, and academic importance have also been addressed. This chapter sets the stage for the problem statement described in the next chapter.

Chapter 2
Problem Definition

2.1 Problem Statement

Currently, when ordering food from restaurants, consumers face various problems such as having little access to menus, inadequate ordering services, delays in food deliveries, and no tracking of delivery processes. Also, many small-scale restaurants do not have a digital presence on the internet making it difficult for them to market their products and services. Some online ordering platforms also have other shortcomings such as expensive services, poor performance in terms of food delivery, and ineffective consumer services. These problems affect both consumers, restaurateurs, and food delivery personnel. Thus, there is a need for an advanced online platform that will offer better solutions to these challenges.

2.2 Proposed Solution Overview

This proposed system is an online food ordering and delivery system that would help to ease the connection between the customer, the restaurants, and the delivery staff. Such systems would include functionalities like registration for users, searching for the desired restaurants, menu management, placing orders, and order delivery tracking. There is going to be security of transactions in addition to a chatbot feature to assist customers. This system uses modern technology and is designed responsively, meaning that customers will be able to access it from various devices. Meals can be ordered and delivered through this system; hence, the whole process will be automated.

2.3 Objectives of the Proposed System

Business Objectives (BO):

- BO-1: Develop a web-based system for online food ordering and delivery.
- BO-2: Improve user experience through a responsive and user-friendly interface.
- BO-3: Provide real-time order tracking and status updates.
- BO-4: Ensure secure authentication and payment processing.
- BO-5: Automate restaurant and delivery management processes.

2.4 Scope of the System

Scope of the Online Food Ordering Delivery System is that the system would be developed to offer a web interface that helps customers to search for restaurants, choose meals, place their orders,

and monitor deliveries instantly. Various user types such as customers, restaurant proprietors, delivery persons, and system administrators can log into the system using different login details. The primary functions of the system include authentication, restaurants and menu management, order processing, payments, and customer feedback options. The online system works on web interfaces and offers responsive access on various devices such as computers and mobile phones.

2.5 System Architecture and Modules

The design of the architecture is based on the client/server architecture pattern with the implementation of a three-layer architecture consisting of the presentation layer, application layer, and database layer. The frontend works as an interface enabling users to interact with the system, whereas the backend does all the business logic handling as well as communication between the system and APIs. Meanwhile, the database tier stores all information managed by the system. The system adopts the modularity technique where it is divided into separate modules such as user, order, and delivery modules.

2.5.1 Module 1: Profile Management

Profile Management can be utilized in managing the procedures related to user registration, login, and user account management.This makes it possible to register the users, update their profiles, and protect their credentials. The application software supports user authentication using JWT and other methods. Different kinds of users include customers, restaurant managers, delivery personnel, and administration. **Functional Requirements:**

- FE-1: The system shall allow users to register using valid credentials.
- FE-2: The system shall authenticate users before granting access.

2.5.2 Module 2: Communication Module

Module: Communication This module helps enable communication between the user and the system. This module is designed with the help of various components including notifications, chatbots, and updates. Notifications are delivered to users about the status of their orders. Users can also get assistance from the chatbot. **Functional Requirements:**

- FE-1: The system shall enable secure communication through notifications.
- FE-2: The system shall generate real-time order updates.
- FE-3: The system shall maintain communication logs.

2.5.3 Module 3: Core Functional Module

This module is at the heart of the system where users browse through various restaurants, view their menus, place orders, and track delivery. This module will ensure that user requests are handled efficiently and that the data entered is handled correctly. This module directly tackles the major problem.

Functional Requirements:

- FE-1: The system shall allow users to browse restaurants and menus.
- FE-2: The system shall process and store orders efficiently.
- FE-3: The system shall provide real-time order tracking.

2.5.4 Module 4: Management / Admin Module

The Admin module provides control and monitoring functionalities. Administrators can manage users, restaurants, and orders, as well as monitor system performance. It also includes reporting tools to analyze system usage and performance.

Functional Requirements:

- FE-1: The system shall allow administrators to manage user accounts.
- FE-2: The system shall generate system usage reports.

2.5.5 Module 5: Advanced / Additional Features

The current module has the following advanced capabilities: payment gateway, chat bot, and ratings systems. The system usability and user experience are improved through provision of a secure way of payment, ratings, and feedback features.

Functional Requirements:

- FE-1: The system shall integrate secure payment gateways.
- FE-2: The system shall allow users to rate and review services.

2.6 Assumptions and Dependencies

Assumptions The system assumes that users have a consistent internet connection and devices such as smartphones or personal computers that are capable of connecting to the internet. The system also assumes that the users have some knowledge about using the system as they must be digitally literate. The system relies on third-party providers for functions such as payments and hosting. In addition, it needs a reliable server on the backend to process any request and transaction.

2.7 Chapter Summary

In this chapter, we have described the problem that has been identified for which the system has been designed. Objectives of the project have been clearly described for meeting the identified problems. Scope and architecture of the system have been discussed here in order to provide structural and functional boundaries to the software. Assumptions and dependency factors have also been included in this chapter.

This chapter forms the basis of the problem and guides the software design in the next chapter.

Chapter 3
Requirement Analysis

3.1 Introduction to Requirement Analysis

This chapter gives a systematic and formal representation of system requirements that will be used in the proposed project. The aim of this chapter is to define the objectives of the system prior to designing the system. The requirements listed in this chapter provide the basis for implementation, verification, and testing of the system.

Requirement analysis comprises requirements gathering methods, stakeholder analysis, use case modeling, functional requirements, non-functional requirements, and external interface requirements. All requirements are written in measurable and testable format using standard requirement engineering principles.

3.2 Requirement Elicitation Techniques

This section discusses how system requirements were gathered and analyzed. Elicitation of system requirements is done to ensure that the stakeholders' requirements are met by the system.

Requirements for this project were identified using the following techniques:

- Stakeholder interviews to capture user expectations and constraints.
- Literature review of similar food ordering systems and research contributions.
- Analysis of existing platforms such as Foodpanda, Uber Eats, and Zomato to identify functional gaps.
- Observation of food ordering workflows and delivery processes.
- Feasibility study to evaluate technical and operational constraints.

3.3 User Classes and Characteristics

In this part, various types of users who interact with the system have been specified. The definition of each type of user, their roles, permissions, and skill level have been mentioned in detail.

3.4 System Overview

The proposed system is supposed to work under the client-server paradigm with the application of the three-tier architecture. Three tiers include the presentation tier, the application tier, and the database tier. The presentation tier offers a user-friendly interface, which can be used by any web browser. All operations with the user's orders and delivering services will be carried out at

Table 3.1: User Classes and Characteristics

User Class	**Description**	**Technical Level**
Administrator	Manages system configuration, user accounts, restaurants, orders, and overall system monitoring.	Advanced
Customer	Browses restaurants, views menus, places orders, tracks deliveries, and provides feedback.	Basic
Restaurant Owner	Manages restaurant profile, menu items, pricing, availability, and order handling.	Intermediate
Delivery Personnel	Handles assigned deliveries, updates delivery status, and interacts with order tracking system.	Intermediate
External System	Payment gateways and third-party APIs integrated for transactions and services.	Technical

the application tier. Information about the users, the restaurants, the orders, and their reviews will be stored at the data tier.

All components of the proposed system will be created following the module-based approach, which implies that all components of the system (i.e., user management, restaurant management, ordering, payment, and deliveries) will be developed independently of each other. All communications among the modules will occur via RESTful APIs.

3.5 Functional Requirements

3.5.1 User Management

- The system shall allow users to register and create accounts.
- The system shall authenticate users using secure login mechanisms.
- The system shall support role-based access control.
- The system shall allow users to update their profiles.

3.5.2 Restaurant Management

- The system shall allow restaurant owners to register their restaurants.
- The system shall allow adding, editing, and deleting menu items.
- The system shall allow updating restaurant details such as location and timings.

3.5.3 Order Management

- The system shall allow customers to browse restaurants and menus.
- The system shall allow customers to place orders.

- The system shall generate order confirmations.
- The system shall provide real-time order tracking.

3.5.4 Payment Processing

- The system shall support online payment methods.
- The system shall support cash on delivery.
- The system shall securely process and store payment information.

3.5.5 Delivery Management

- The system shall assign delivery personnel to orders.
- The system shall allow delivery personnel to update delivery status.
- The system shall provide real-time delivery tracking to customers.

3.5.6 Rating and Reviews

- The system shall allow customers to rate restaurants.
- The system shall allow customers to submit reviews.
- The system shall display ratings for transparency.

3.6 Non-Functional Requirements

- Performance: The system shall respond within a few seconds under normal load.
- Security: The system shall use secure authentication and encrypted communication.
- Usability: The system shall provide an intuitive and user-friendly interface.
- Availability: The system shall be available 24/7 with minimal downtime.
- Scalability: The system shall support increasing number of users and transactions.
- Compatibility: The system shall support modern web browsers and devices.

3.7 External Interface Requirements

3.7.1 User Interface

The web application is provided with a responsive user interface which can be accessed via desktops and mobile phones. The platform offers dashboards, forms, and other interactive components for all types of users.

3.7.2 Hardware Interface

The system requires standard computing devices such as laptops, desktops, tablets, or smartphones with internet connectivity.

3.7.3 Software Interface

The system integrates with:

- Payment gateways (e.g., Stripe)
- Web browsers (Chrome, Firefox, Edge)
- Backend server and database (Node.js and MongoDB)

3.7.4 Communication Interface

The system uses HTTP/HTTPS protocols and RESTful APIs for communication between client and server, ensuring secure and efficient data exchange.

3.8 Use Case Model

The use case model represents high level system interactions between actors and system functionalities. It defines system boundaries and captures functional behavior from a user perspective.

3.8.1 Use Case Diagram

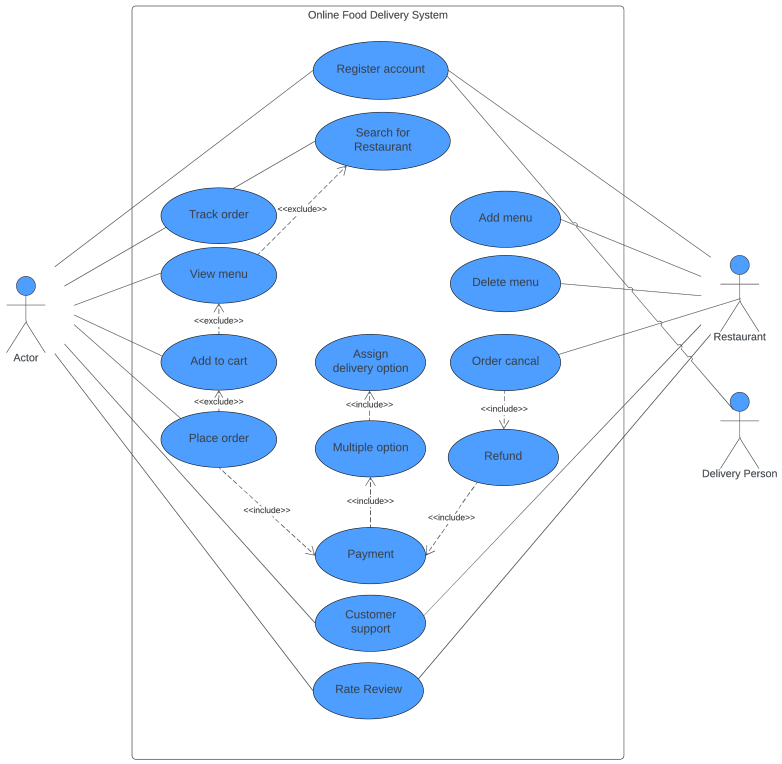

Figure 3.1: Use Case Diagram

The above diagram illustrates the interaction between actors and major system functionalities. Include relationships such as *include* and *extend* where applicable.

Table 3.2: Register Account

Use Case ID:	**UC-1**
Use Case Name:	Register Account
Actors:	Customer, Restaurant manager, Delivery personnel
Description:	A Customer, manager and Delivery Personnel registers for a new account in the online food delivery system.
Preconditions:	None
Postconditions:	New account is created and stored in the system.
Normal Flow:	1. Customer selects the option to register a new account. 2. System displays the registration form. 3. Customer enters details and submit the form.
Alternative Flows:	None

Table 3.3: Use Case: Search for Restaurant

Use Case ID:	**UC-2**
Use Case Name:	Search for Restaurant
Actors:	Primary Actor: Customer
Description:	A Customer searches for restaurants in the online food delivery system.
Preconditions:	Customer is logged into the system.
Postconditions:	System displays a list of restaurants matching the search criteria.
Normal Flow:	1. Customer enters search criteria. 2. System displays a list of restaurants that match the search criteria.
Alternative Flows:	None

Table 3.4: Use Case: View Menu

Use Case ID:	**UC-3**
Use Case Name:	View Menu
Actors:	Primary Actor: Customer
Description:	A Customer views the menu of a selected restaurant.
Preconditions:	Customer has selected a restaurant.
Postconditions:	System displays the menu of the selected restaurant.
Normal Flow:	1. Customer selects the option to view the menu of a selected restaurant. 2. System displays the menu items and prices.
Alternative Flows:	None

Table 3.5: Use Case: Add to Cart

Use Case ID:	**UC-4**
Use Case Name:	Add to Cart
Actors:	Primary Actor: Customer
Description:	A Customer adds selected menu items to their cart.
Preconditions:	Customer is viewing the menu of a selected restaurant.
Postconditions:	Selected menu items are added to the Customer's cart.
Normal Flow:	1. Customer selects menu items to add to the cart. 2. System updates the cart with selected menu items.
Alternative Flows:	None

Table 3.6: Use Case: Place Order

Use Case ID:	**UC-5**
Use Case Name:	Place Order
Actors:	Primary Actor: Customer
Description:	A Customer places an order for the items in their cart.
Preconditions:	Customer has items in their cart.
Postconditions:	Order is placed and stored in the system. Delivery schedule is updated.
Normal Flow:	1. Customer reviews the items in the cart. 2. Customer proceeds to checkout. 3. Customer selects delivery options and specifies the delivery address. 4. Customer confirms the order. 5. System processes the order and provides an order confirmation.
Alternative Flows:	None

Table 3.7: Use Case: Assign Delivery Option

Use Case ID:	**UC-6**
Use Case Name:	Assign Delivery Option
Actors:	Primary Actor: Customer
Description:	A Customer assigns delivery options for their order.
Preconditions:	Customer is in the process of placing an order.
Postconditions:	Delivery options are assigned to the order.
Normal Flow:	1. Customer specifies the delivery address.
Alternative Flows:	None

Table 3.8: Use Case: Payment

Use Case ID:	**UC-7**
Use Case Name:	Payment
Actors:	Primary Actor: Customer
Description:	A Customer makes a payment for their order.
Preconditions:	Customer has confirmed the order.
Postconditions:	Payment is processed and confirmed. Order status is updated to "Paid."
Normal Flow:	1. Customer selects a payment method. 2. Customer enters payment details. 3. System processes the payment. 4. System confirms the payment and updates the order status.
Alternative Flows:	None

Table 3.9: Use Case: Customer Support

Use Case ID:	**UC-8**
Use Case Name:	Customer Support
Actors:	Primary Actor: Customer
Description:	A Customer contacts customer support for assistance.
Preconditions:	None
Postconditions:	Customer's issue is logged and addressed.
Normal Flow:	1. Customer selects the option to contact customer support. 2. System displays contact options (e.g., chat, email, phone). 3. Customer chooses a contact method and describes their issue.
Alternative Flows:	None

Table 3.10: Use Case: Rate Review

Use Case ID:	**UC-9**
Use Case Name:	Rate Review
Actors:	Primary Actor: Customer
Description:	A Customer rates and reviews their order experience.
Preconditions:	Customer has received their order.
Postconditions:	Customer's rating and review are stored in the system.
Normal Flow:	1. Customer selects the option to rate and review their order. 2. Customer enters their rating and review. 3. Customer submits the rating and review. 4. System stores the rating and review and displays a confirmation.
Alternative Flows:	None

Table 3.11: Use Case: Track Order

Use Case ID:	**UC-10**
Use Case Name:	Track Order
Actors:	Primary Actor: Customer
Description:	A Customer tracks the status of their order.
Preconditions:	Customer has placed an order.
Postconditions:	Customer is informed of the current status of their order.
Normal Flow:	1. Customer selects the option to track their order. 2. System displays the current status.
Alternative Flows:	None

Table 3.12: Use Case: Add Menu

Use Case ID:	**UC-11**
Use Case Name:	Add Menu
Actors:	Primary Actor: Restaurant
Description:	A Restaurant adds menu items to their online menu.
Preconditions:	Restaurant is logged into the system.
Postconditions:	New menu items are added to the restaurant's online menu.
Normal Flow:	1. Restaurant selects the option to add menu items. 2. System displays the add menu form. 3. Restaurant enters details of new menu items (e.g., name, description, price). 4. Restaurant submits the form.
Alternative Flows:	None

Table 3.13: Use Case: Delete Menu

Use Case ID:	**UC-12**
Use Case Name:	Delete Menu
Actors:	Primary Actor: Restaurant
Description:	A Restaurant deletes menu items from their online menu.
Preconditions:	Restaurant is logged into the system.
Postconditions:	Selected menu items are removed from the restaurant's online menu.
Normal Flow:	1. Restaurant selects the option to delete menu items. 2. Restaurant selects menu items to delete. 3. System validates the request and updates the online menu.
Alternative Flows:	None

3.9 Functional Requirements

Functional requirements define the core services and behaviors that the system shall provide. These requirements are derived directly from the use case model presented in Section 3.5. Each requirement is uniquely identified, measurable, and written using formal specification language.

All functional requirements follow IEEE-style structure and are categorized by system modules.

1. **User Authentication and Authorization:**
    - User can register, login, and logout.
    - Authentication mechanisms like JWT tokens.
2. **Restaurants Management:**
    - Restaurant registration and profile management.
    - Menu management (add, edit, delete items, prices, descriptions, images).
    - Operating hours, location, and delivery zone management.
3. **Ordering System:**
    - Browse restaurants and menus.
    - Customer can view menus.
    - Customer can place orders.
    - Real-time order status tracking.
4. **Payment Processing:**
    - Multiple payment methods (Stripe, cash on delivery).
    - Secure payment card data handling (compliance with security standards)[**?**].
5. **Delivery Management:**
    - Assign delivery personnel to orders.
    - Real-time delivery status tracking.
6. **Rating and Reviews:**
    - Customer ratings and reviews for restaurants and delivery experiences.
    - Use ratings and reviews to enhance platform credibility and trustworthiness.
7. **Customer Support:**
    - Customer support channels (live chat or email, phone).
    - FAQ section.
8. **Responsive Design:**

- Responsive design for various devices (smartphones, tablets, desktops).
- Use responsive web design techniques and frameworks like Bootstrap.

9. **Chatbot:**
    - A simple custom chatbot for customer support.
    - Chatbot answering the user questions[**?**].

3.10 Non-Functional Requirements

3.10.1 Reliability

- The system should be reliable and available at all times, with minimal downtime or maintenance.
- If something goes wrong, it should be fixed quickly.
- Users should feel confident that the website will work as expected.

3.10.2 Usability

- System is expected to be simple even for the first timers in the system.
- Users of the system must be able to use it easily and have access to the information without much struggle.
- It has to help clients step by step in the order they are supposed to go or follow.
- The system should also be amplitude and in low latencies so that clients will not waste time and will find it simple to make their orders.

3.10.3 Performance

- The webApp has to be responsive and fast.
- Pages need to get up in not more than the four seconds mark.
- The system must be able to enable a multitude of users to execute tasks in the system at once.
- The system has to be able to receive and process a number of orders so that the clients do not have to wait long for their food[**?**].

3.10.4 Security

- Everyone's information will be well safeguarded using advanced encryption technology.
- The system has to guarantee safe reliable and trustworthy payment means.

3.11 External Interface Requirements

In this section, an account of how the system connects with the external environment is given based on user interaction with hardware, software, and communications.

3.11.1 User Interface Requirements

- UI-1: The system shall provide a graphical user interface (GUI) for all user roles.
- UI-2: The interface shall use standard and readable font sizes (12–14pt) for clarity.
- UI-3: The layout shall be responsive and adaptable across desktop, tablet, and mobile devices.
- UI-4: Navigation shall remain consistent across all modules to ensure ease of use.
- UI-5: The system shall provide clear error messages and feedback for user actions.
- UI-6: The interface shall support intuitive interaction with minimal user training.

3.11.2 Hardware Interface Requirements

- HI-1: The system shall operate on standard computing devices such as desktops, laptops, and smartphones.
- HI-2: For mobile applications, the system shall support Android and iOS platforms.
- HI-3: The system shall require standard input/output devices such as keyboard, mouse, and touchscreen.
- HI-4: For high-performance tasks (e.g., ML processing), GPU support may be utilized where applicable.

3.11.3 Software Interface Requirements

- SI-1: The system shall interact with a database management system (relational or non-relational) for data storage and retrieval.
- SI-2: The system shall integrate securely with third-party APIs where required.
- SI-3: Payment gateways shall be integrated using secure and authenticated protocols.
- SI-4: The system shall utilize frameworks and libraries appropriate to the technology stack used (e.g., web frameworks, ML libraries).
- SI-5: The system shall support interoperability with external services through well-defined APIs.

3.11.4 Communication Interface Requirements

- CI-1: The system shall use HTTPS protocol to ensure secure communication over the network.
- CI-2: RESTful APIs shall be used for communication between client and server.
- CI-3: The system shall support real-time communication using WebSockets where required (e.g., live tracking).
- CI-4: Data exchange between modules shall follow standardized formats such as JSON or XML.
- CI-5: Secure authentication mechanisms (e.g., tokens or session management) shall be used for communication.

3.12 Assumptions and Dependencies

This section lists assumptions made during requirement analysis and system design planning.

- The system assumes stable and continuous internet connectivity for online operations.
- Third-party services such as payment gateways and APIs are assumed to remain available and functional.
- Users are assumed to possess basic digital literacy to operate the system.
- Required hardware resources and software environments are assumed to be available during deployment.
- External services will comply with defined API contracts and response formats.

3.13 Requirement Traceability Matrix

This matrix maps project objectives to functional requirements to ensure alignment and completeness.

Table 3.14: Requirement Traceability Matrix

Objective	**Use Case**	**Functional Requirement**
BO-1	UC-01	FR-1.1
BO-2	UC-04	FR-2.1
BO-3	UC-05	FR-3.1
BO-4	UC-03	FR-1.2

Chapter Summary

In this chapter, an organized requirement analysis was conducted on the proposed system. The requirement elicitation techniques, user categories, use case modeling, functional requirements, non-functional requirements, and external interface requirements were specified. A requirement traceability matrix was provided to correlate the objectives of the system with the requirement specifications. This chapter lays down the requirements that form the basis of design and development in the upcoming chapters.

Chapter 4
Software Design

4.1 Introduction to Software Design

In this chapter, we present the software design of the proposed system. The software design is obtained from translating the identified requirements in chapter three into architectural, behavioral, and data modeling forms. The main aim of this chapter is to give a guideline for implementing the proposed system.

In this chapter, the architectural design, UML models, data flow models, and database design of the proposed system for use in web-based and distributed environments are presented.

4.2 Design Methodology and Software Process Model

4.2.1 Design Methodology

This project follows a **Layered Architecture combined with Model View Controller (MVC)** design approach. The system is divided into distinct layers including the presentation layer, application/business logic layer, and data access layer.

- **Presentation Layer:** Handles user interaction through web or mobile interfaces.
- **Application Layer:** Contains business logic such as order processing, authentication, and validation.
- **Data Layer:** Manages database operations including storage, retrieval, and updates.

The MVC pattern is used to separate the application into three interconnected components:

- **Model:** Represents the data and business logic.
- **View:** Represents the user interface.
- **Controller:** Handles user input and interacts with the model.

This methodology is chosen because it ensures modularity, scalability, and maintainability. It also allows independent development of components, making the system easier to test and extend.

The design follows key software engineering principles such as:

- **Separation of Concerns:** Each module has a distinct responsibility.
- **DRY (Don't Repeat Yourself):** Avoids duplication of logic.
- **SOLID Principles:** Ensures maintainable and extensible code structure.

4.2.2 Software Process Model

The project follows the **Agile Software Development Model**. This iterative and incremental approach is selected because it allows continuous improvement, frequent testing, and adaptability to changing requirements.

- The development process is divided into multiple iterations or sprints.
- Each iteration includes requirement analysis, design, implementation, testing, and review.
- Feedback from each phase is incorporated into the next iteration.
- Continuous integration and testing are performed to ensure system quality.

Agile is suitable for this project because:

- Requirements may evolve during development.
- It supports incremental delivery of system features.
- It enables early detection of issues through continuous testing.
- It improves collaboration between stakeholders and developers.

Testing is integrated into each iteration, including unit testing, integration testing, and system testing, ensuring that each module functions correctly before moving to the next phase.

4.3 Data Flow Diagrams (DFDs)

A DFD or Data Flow Diagram, is a type of diagram that explains how information flows within a network. It outlines the activities data goes through from an input to an output and explains how data is handled within the system. The DFDs are a fundamental part of the design of systems, ensuring that there is an easy comprehension on how information is processed and kept enabling one to improve workflows quickly. DFDs are made in levels such that the next level of a system provides more information about the data flow in the system's processes.

4.3.1 Zero Level DFDs

DFD level 0, as context diagram, is high level view of the system which shows the system as a single pro cess and its externl entities and data flows.

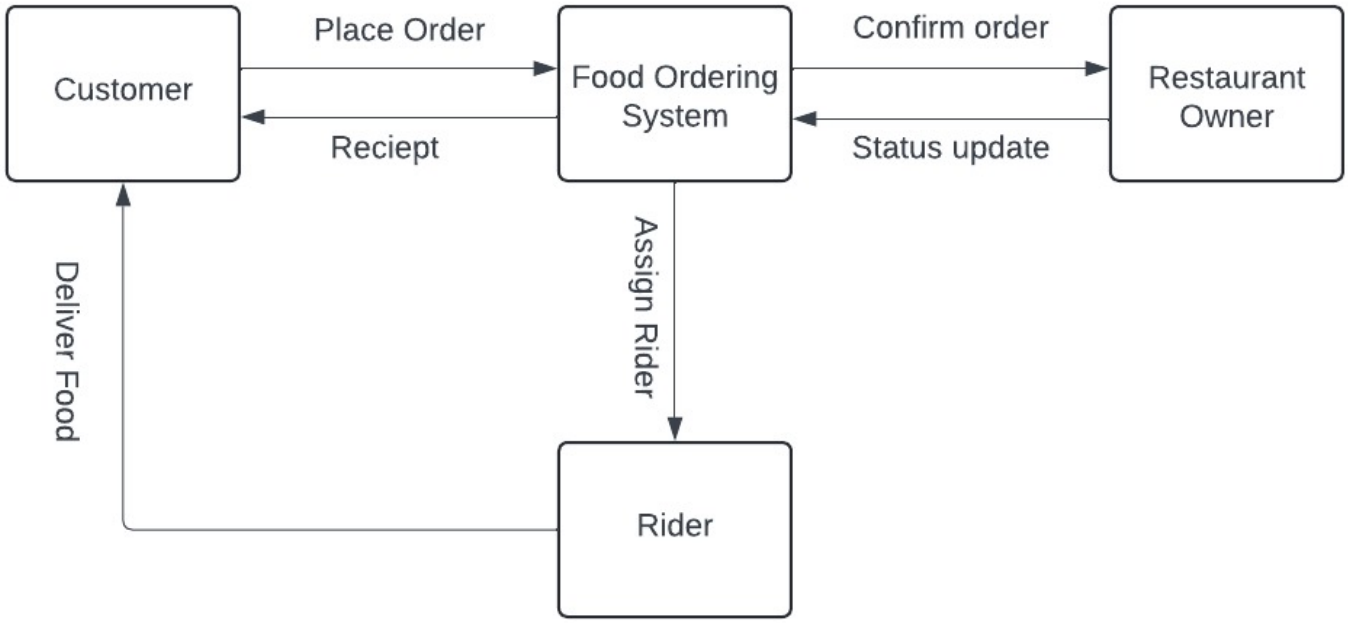

Figure 4.1: Zero Level DFD

4.3.2 First Level DFDs

In DFD Level 1, the single process shown at Level 0 is divided into sub processes, which describe the working of the system.

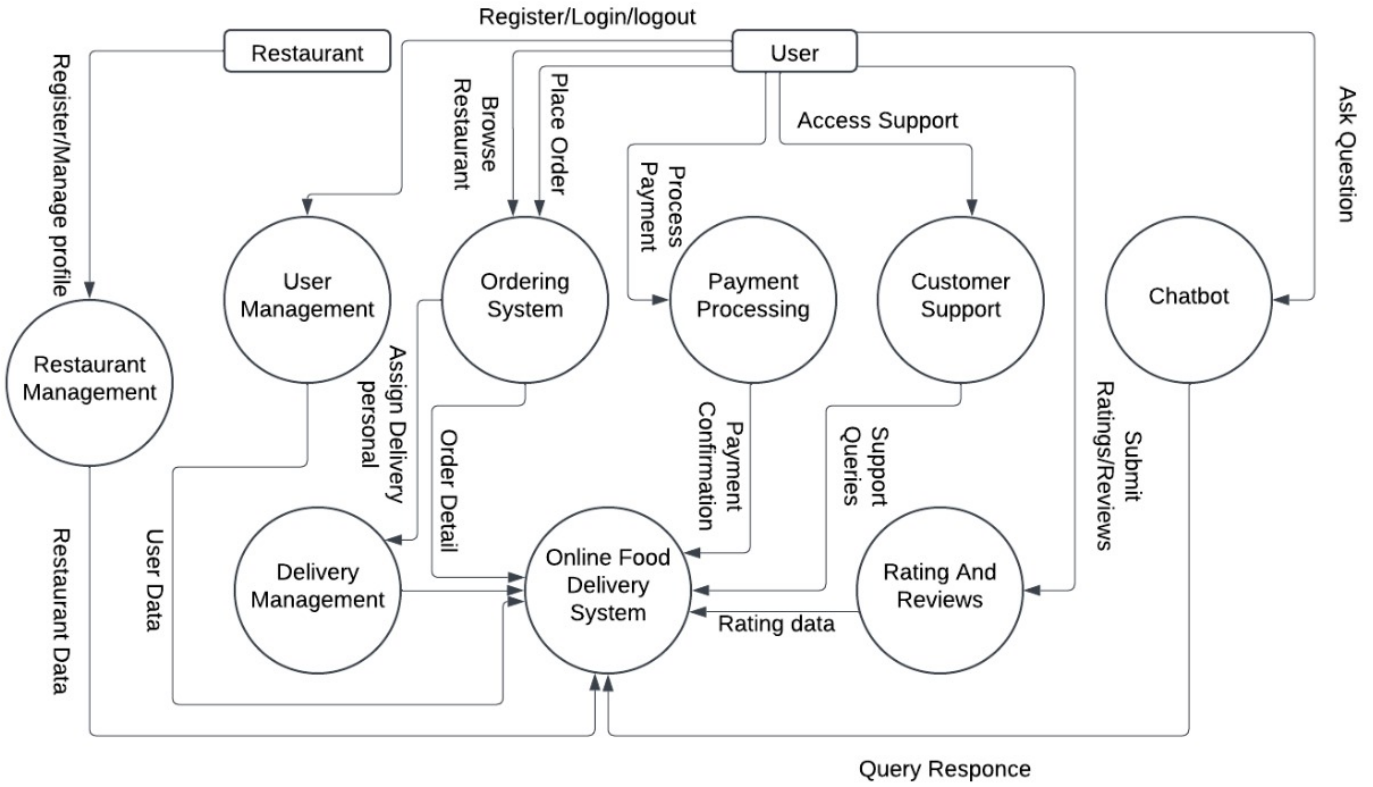

Figure 4.2: First Level DFD

4.3.3 Second Level DFDs

In Level 2 DFDs, we can further break down one of the components, such as the Ordering System, into sub-processes.

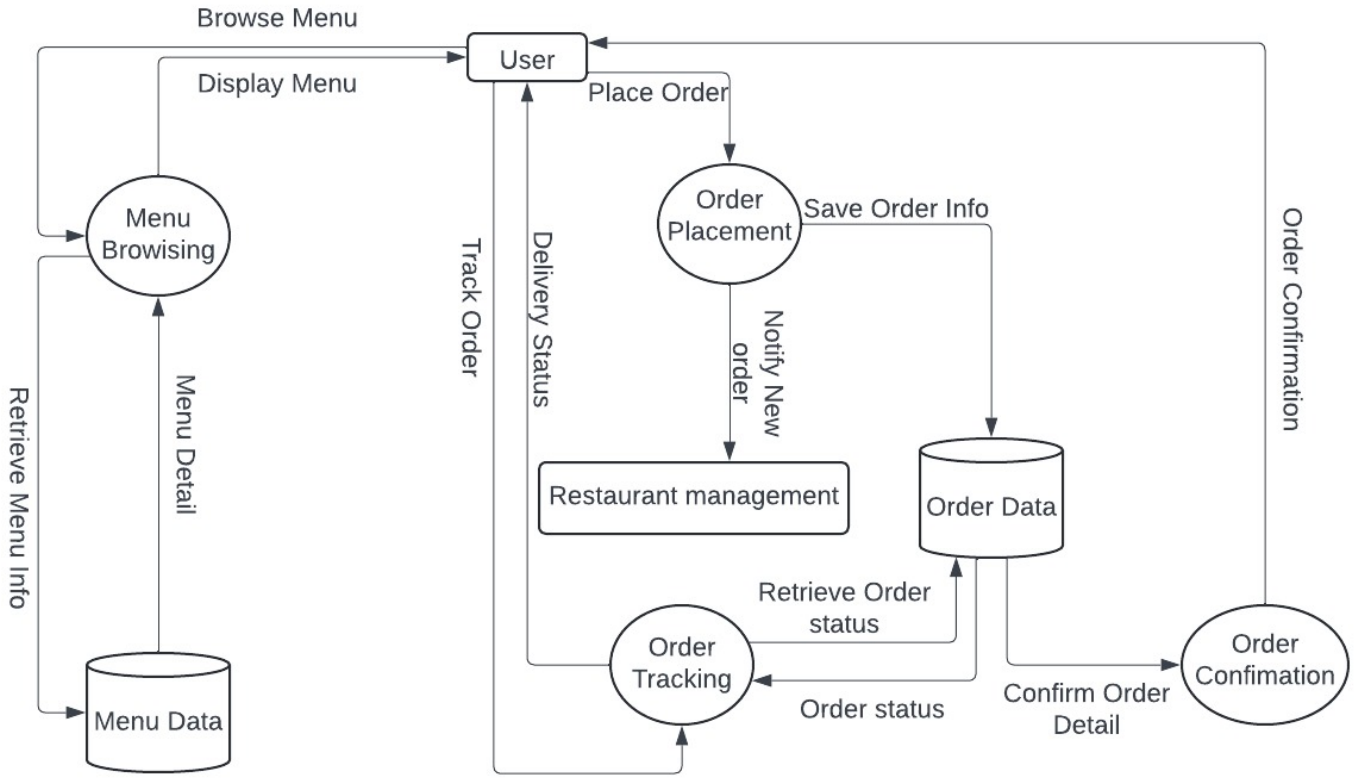

Figure 4.3: Second Level DFD

4.4 Interaction Diagram

The message flow chart represents the way various components exchange messages in order to achieve their goal. With this, it becomes easier for developers to understand the behavior of systems in terms of dynamicity through message exchange between components to accomplish specific operations.

4.4.1 Sequence Diagrams

A sequence diagram is a visual depiction of the ordering of interactions between objects or components in a system over a period of time in order to fulfill a process. It is a message flow shows the sequence of messages between objects and the order of messages, e.g. as user login or order processing. The diagram illustrates what differ ent entities are involved, what messages are sent, and in what time sequence each interaction occurs.

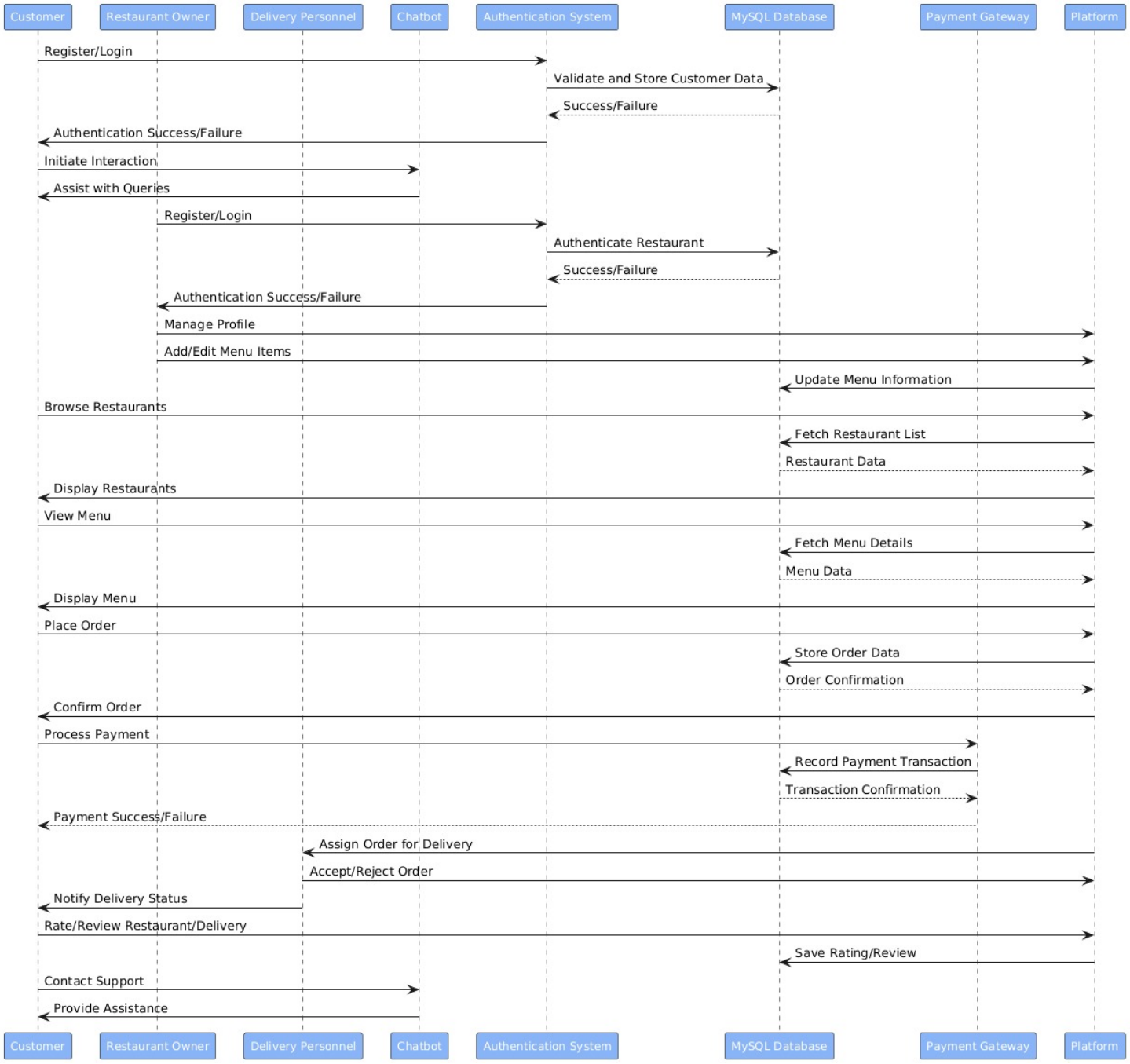

Figure 4.4: Sequence Diagram

4.4.2 ERD

An ERD is used to show the database structure of a system, show that how entities of a tables relate to each other.

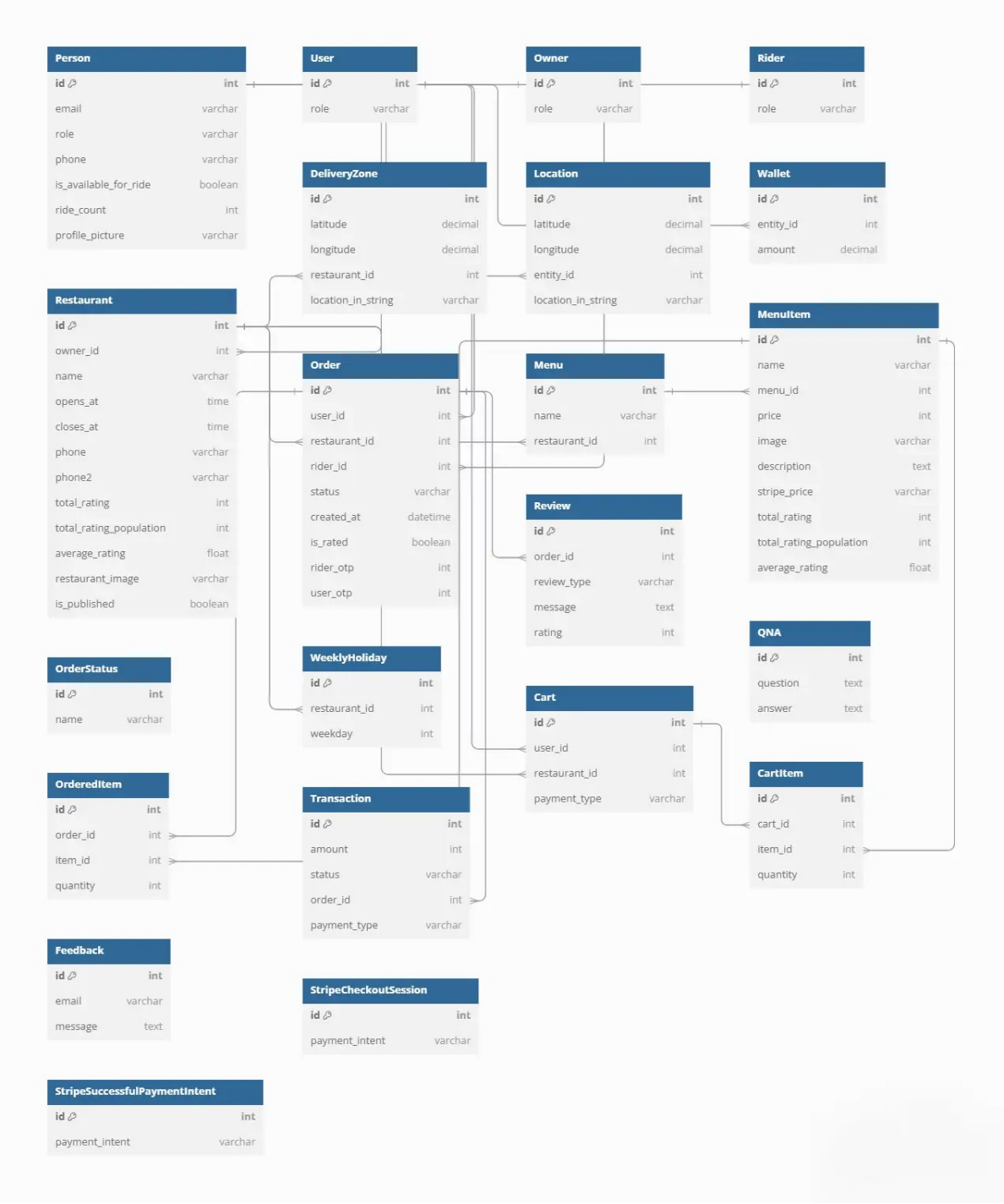

Figure 4.5: Entity-Relationship Diagram (ERD)

Chapter 5
Implementation

5.1 Deployment Requirements

The deployment requirements for the Online Food Ordering and Delivery System is:

1. **Platform Accessibility**
    - Requires an active internet connection for users to access the platform.
    - Optimized for use on various devices, including desktop computers, laptops, tablets, and smartphones.
2. **Browser Compatibility**
    - Compatible with modern web browsers such as Google Chrome, Mozilla Firefox, and Microsoft Edge.
3. **Technical Stack**
    - **Backend:** Python with the Django framework.
    - **Frontend:** HTML, CSS, JavaScript, and Bootstrap for responsive design..
    - **Database:** MySQL for efficient data storage and scalability.
    - **Payment Processing:** Integration with third-party services Stripe for secure payment transactions.
    - **Chatbot:** Botpress is utilized to enable the deployment of Chatbot. Training of Chatbot is performed via our dataset through Botpress. Botpress is an extremely flexible platform, open source in nature, that is employed in building intelligent chatbots with NLU capabilities.
4. **APIs and Integrations**
    - **Payment Processing:** Integration with Stripe.
    - **Map Services:** Google Maps API for showing nearest restaurants and tracking delivery personnel.
5. **Hosting Requirements**
    - **Cloud Hosting:** AWS, Google Cloud, Azure, for scalability and reliability.

5.2 Graphic User Interfaces (GUIs)

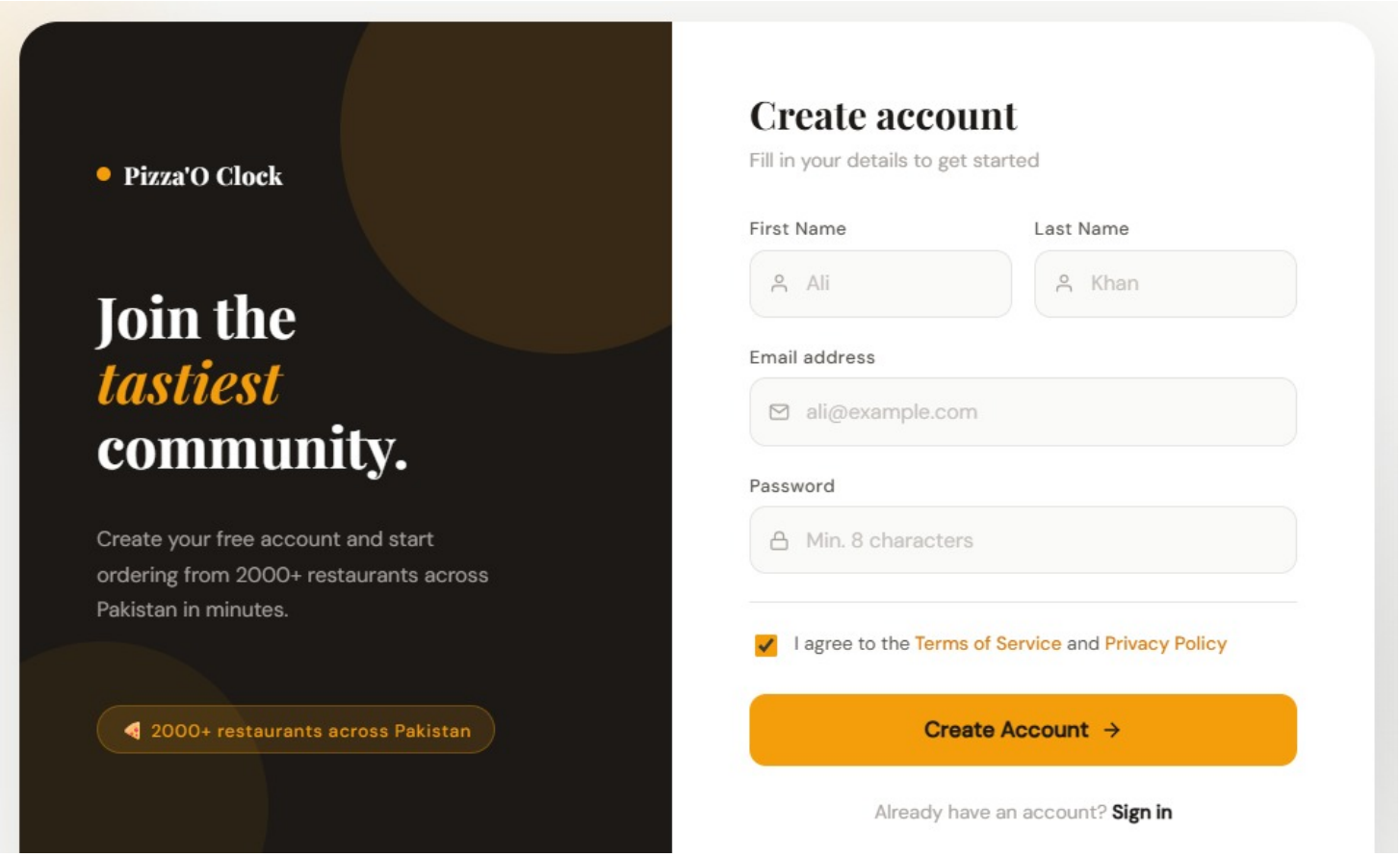

Figure 5.1: Customer Signup

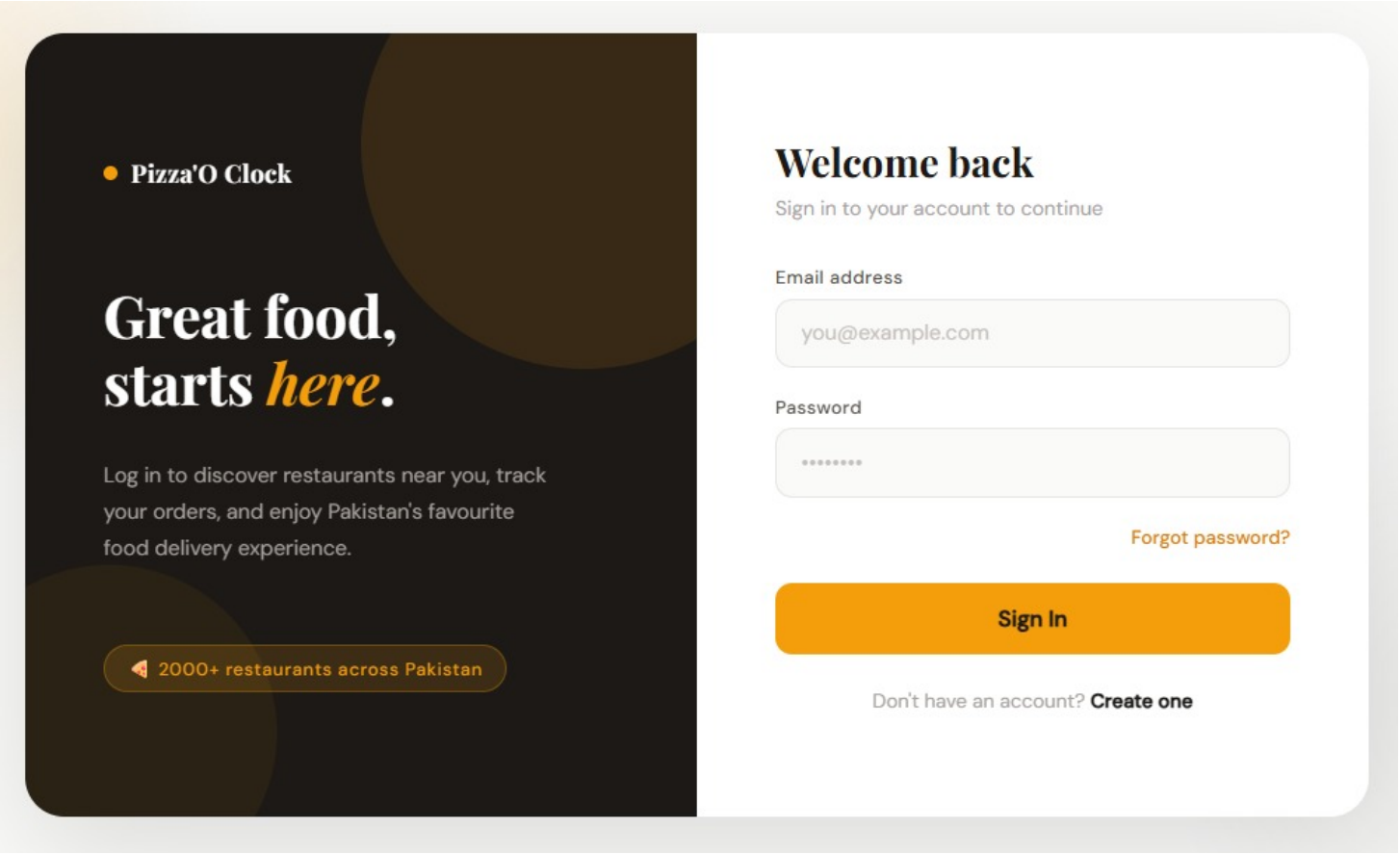

Figure 5.2: Customer Login

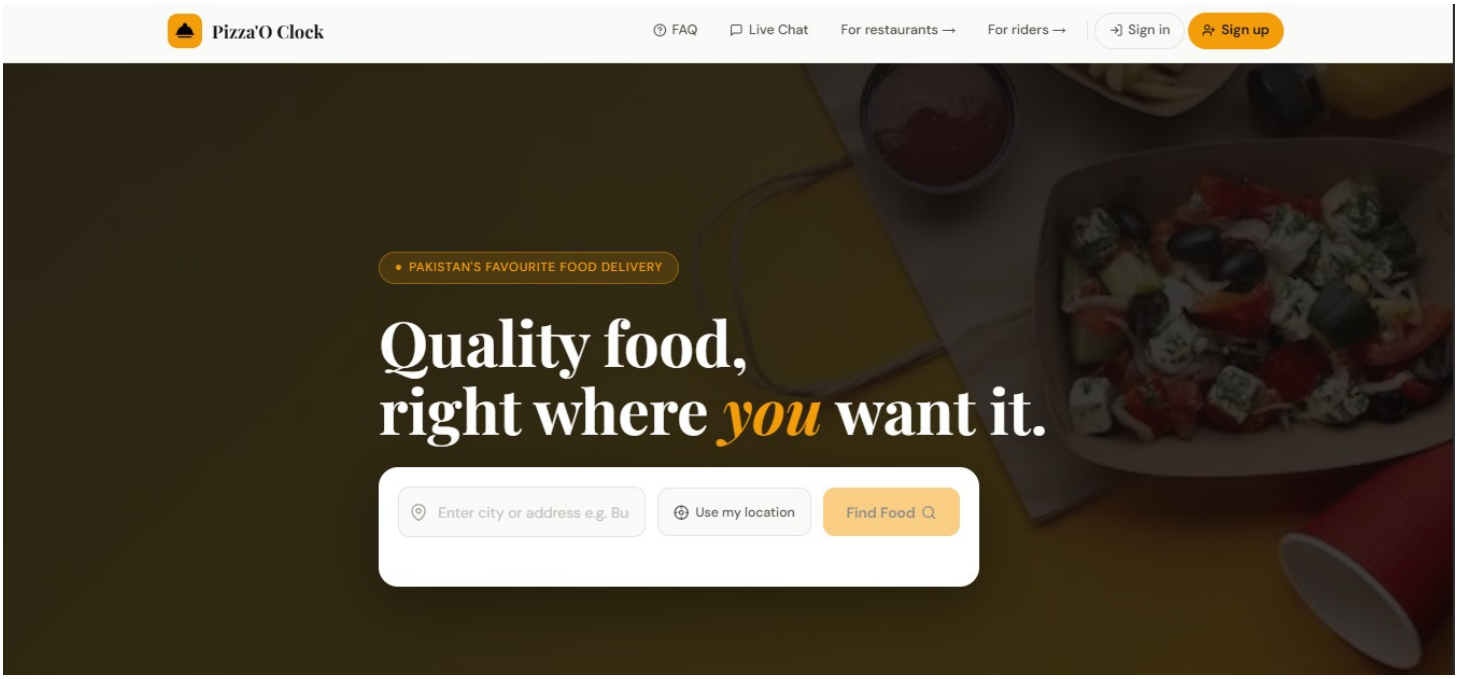

Figure 5.3: Landing Page

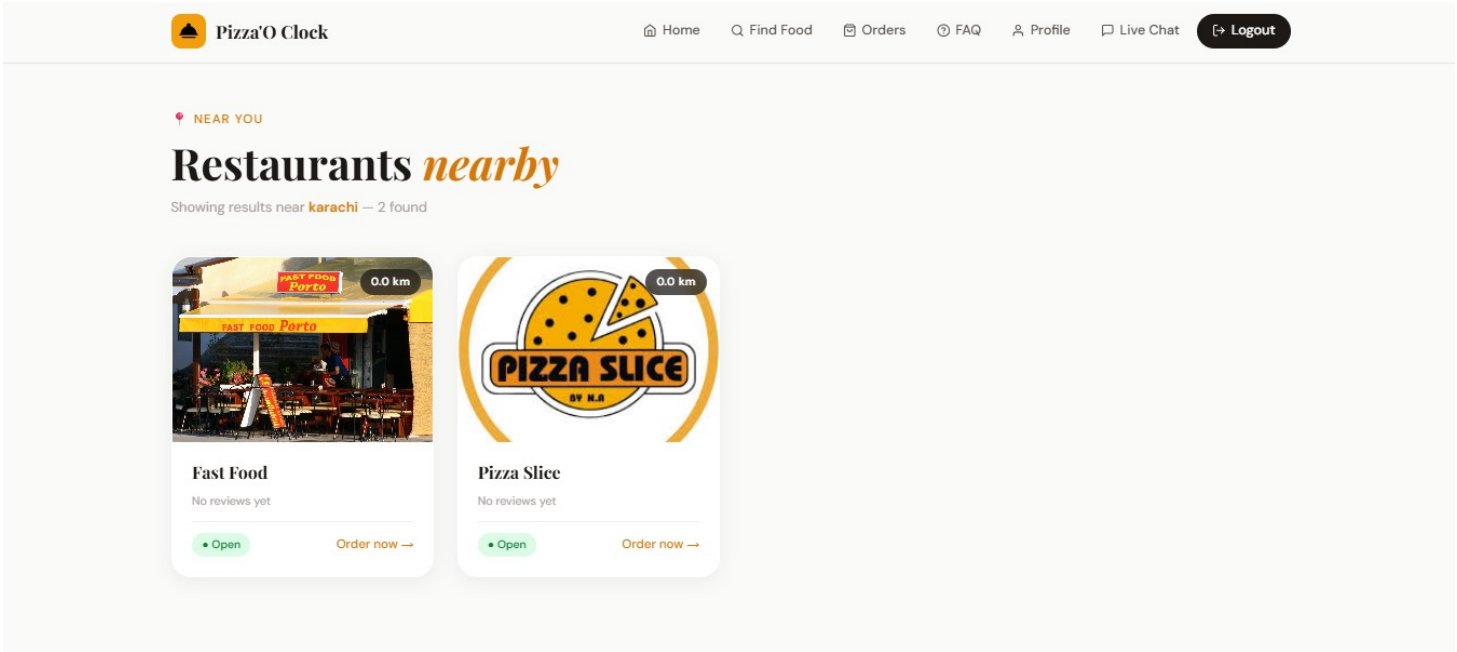

Figure 5.4: Search Restaurants

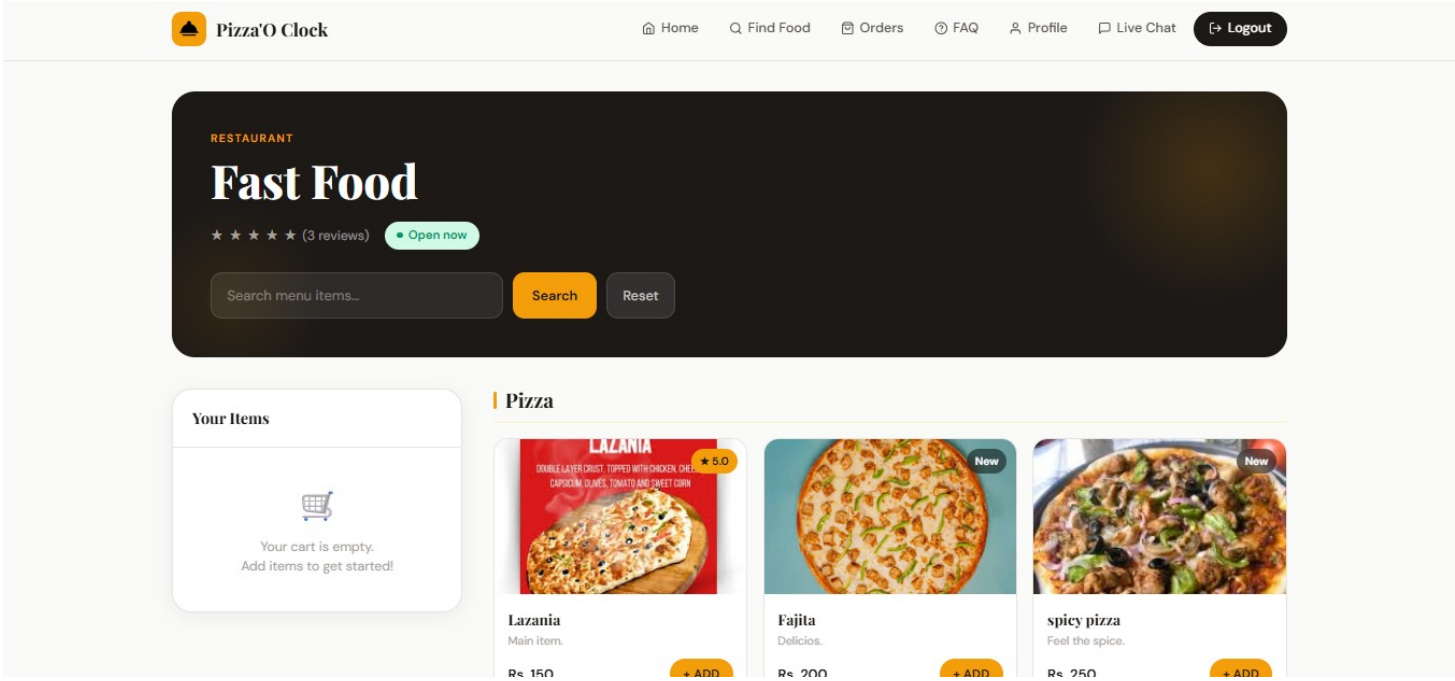

Figure 5.5: Restaurant Menu

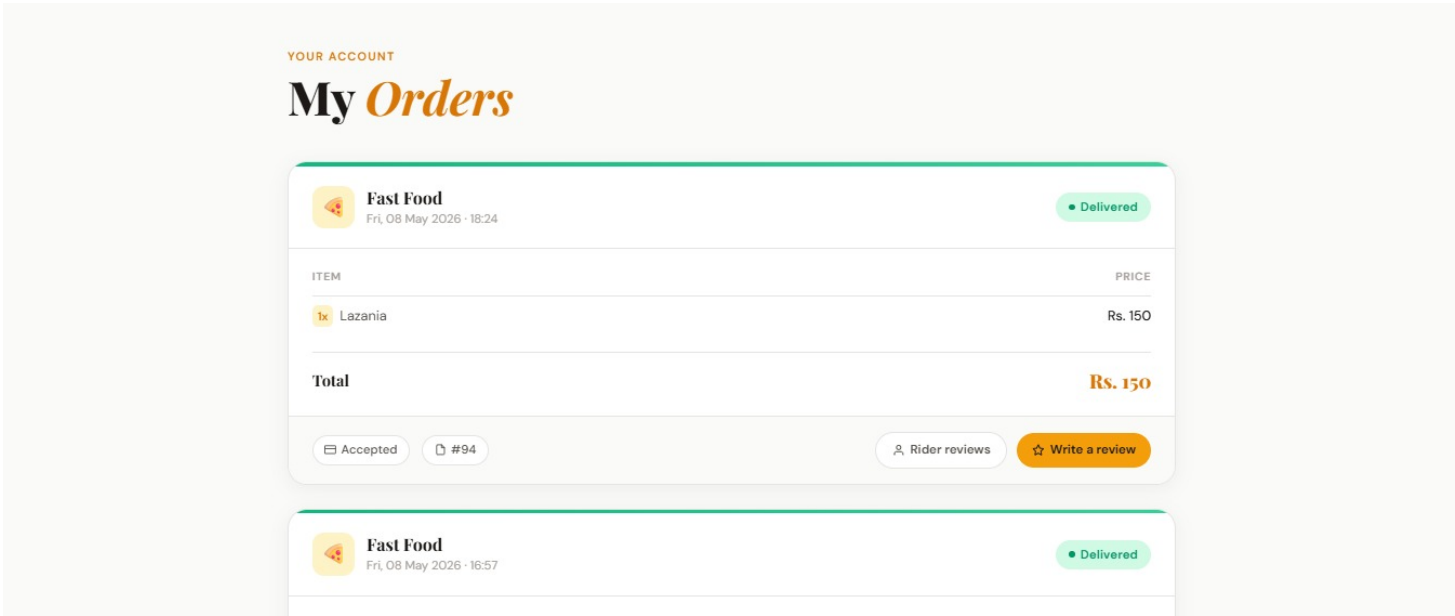

Figure 5.6: Order Track

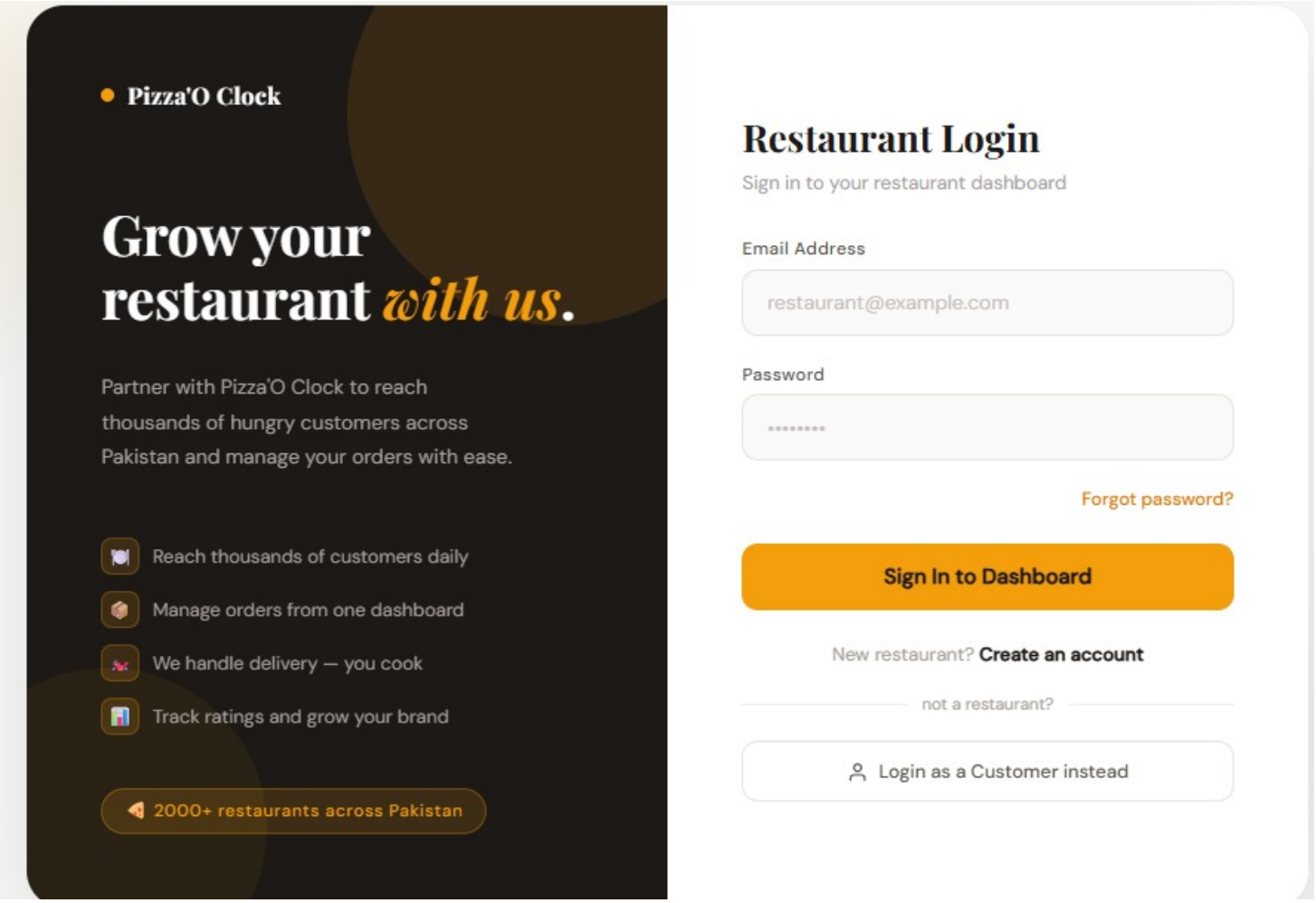

Figure 5.7: Restaurant Login

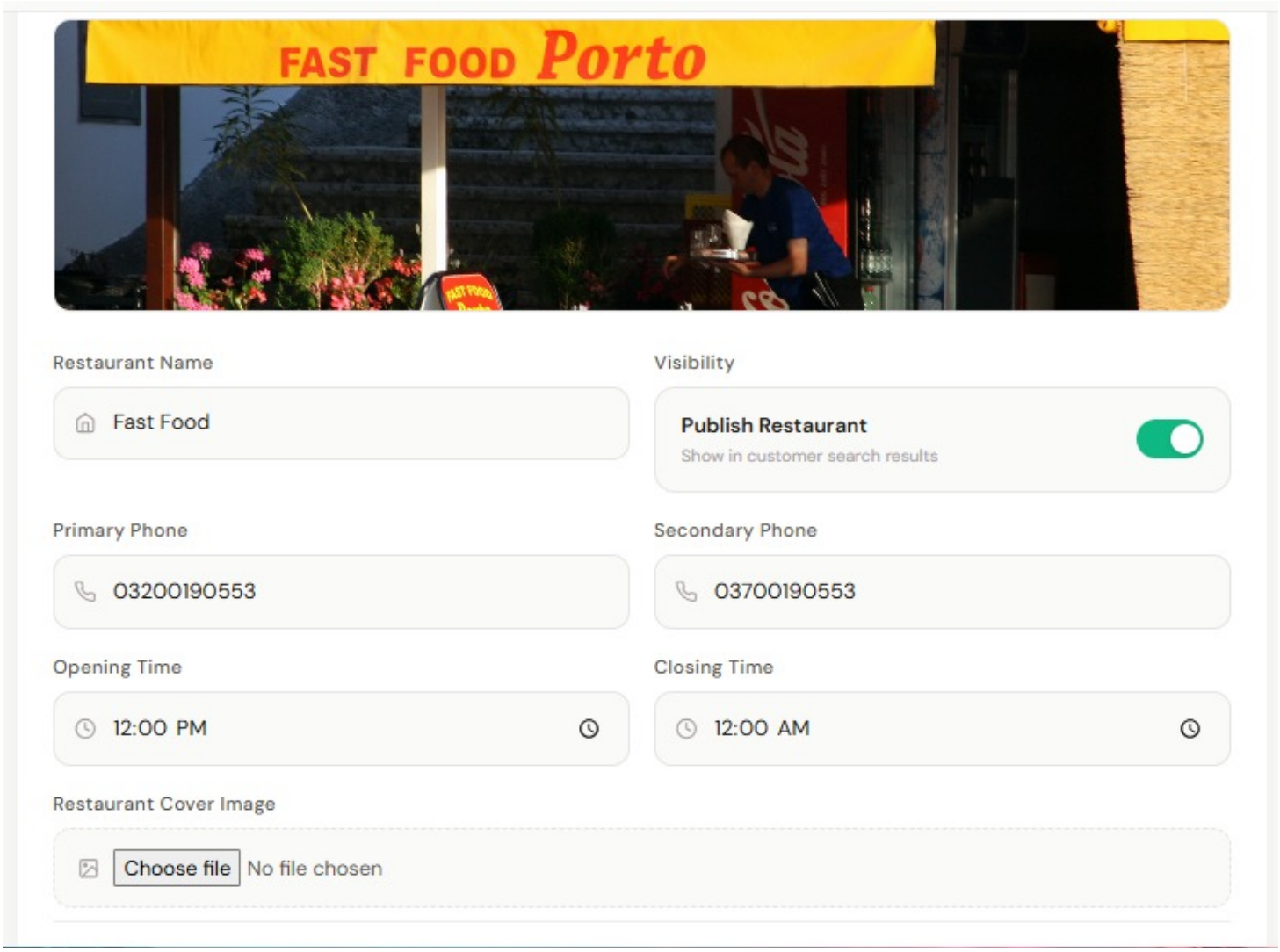

Figure 5.8: Restaurant Registration

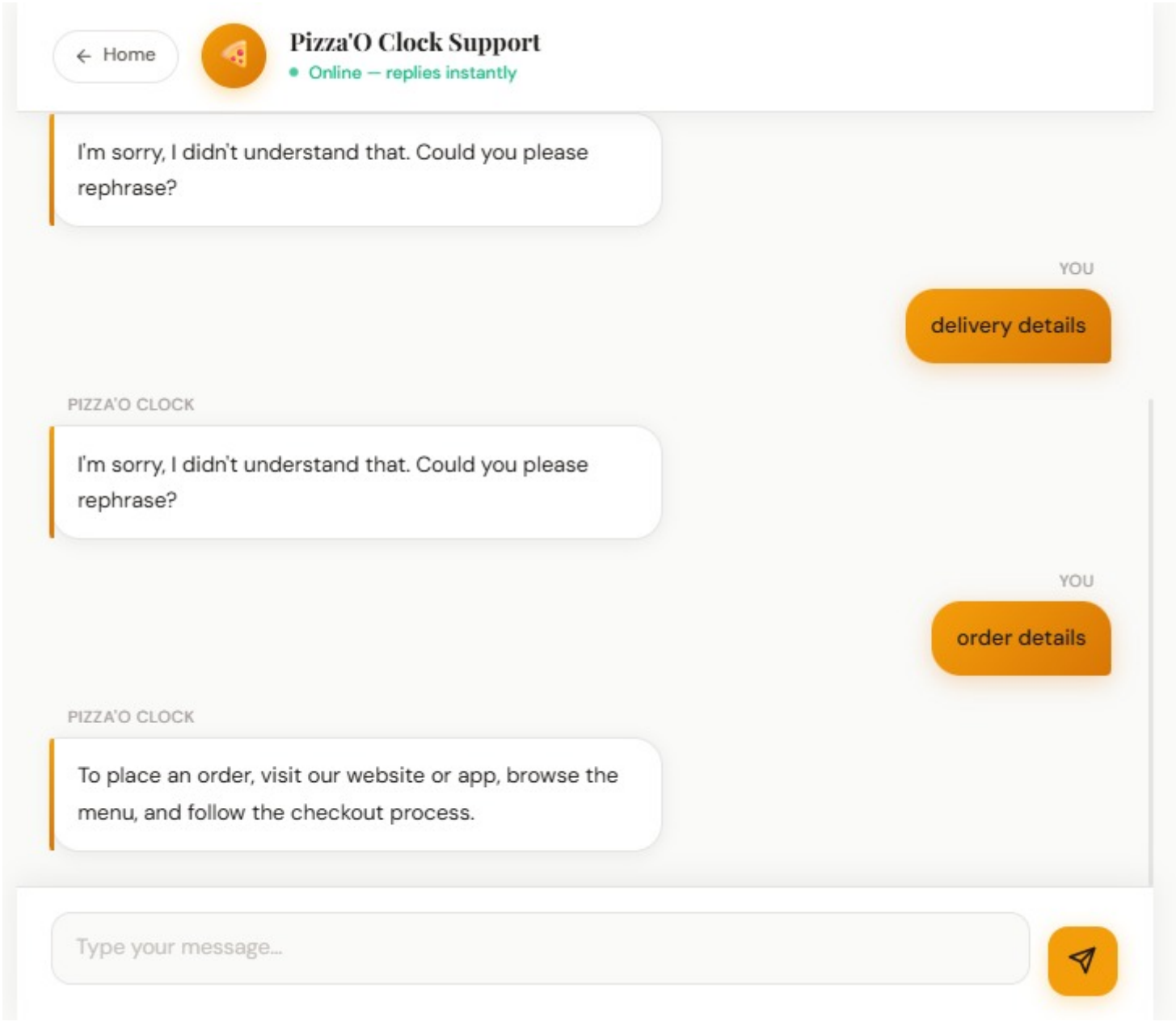

Figure 5.9: Live Chat

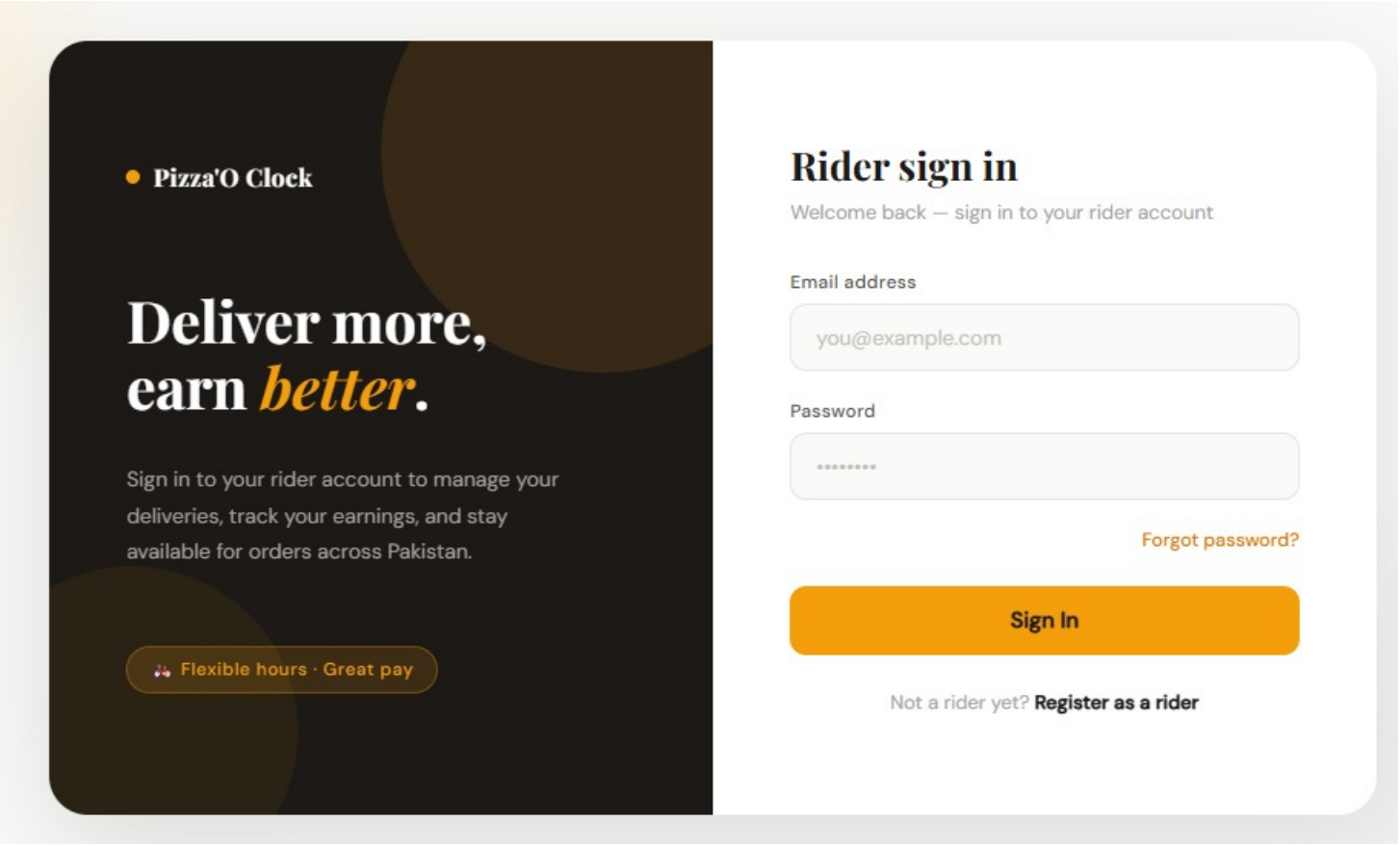

Figure 5.10: Rider Login

5.3 Flowchart

The Flowchart for Online Food Ordering and Delivery website :

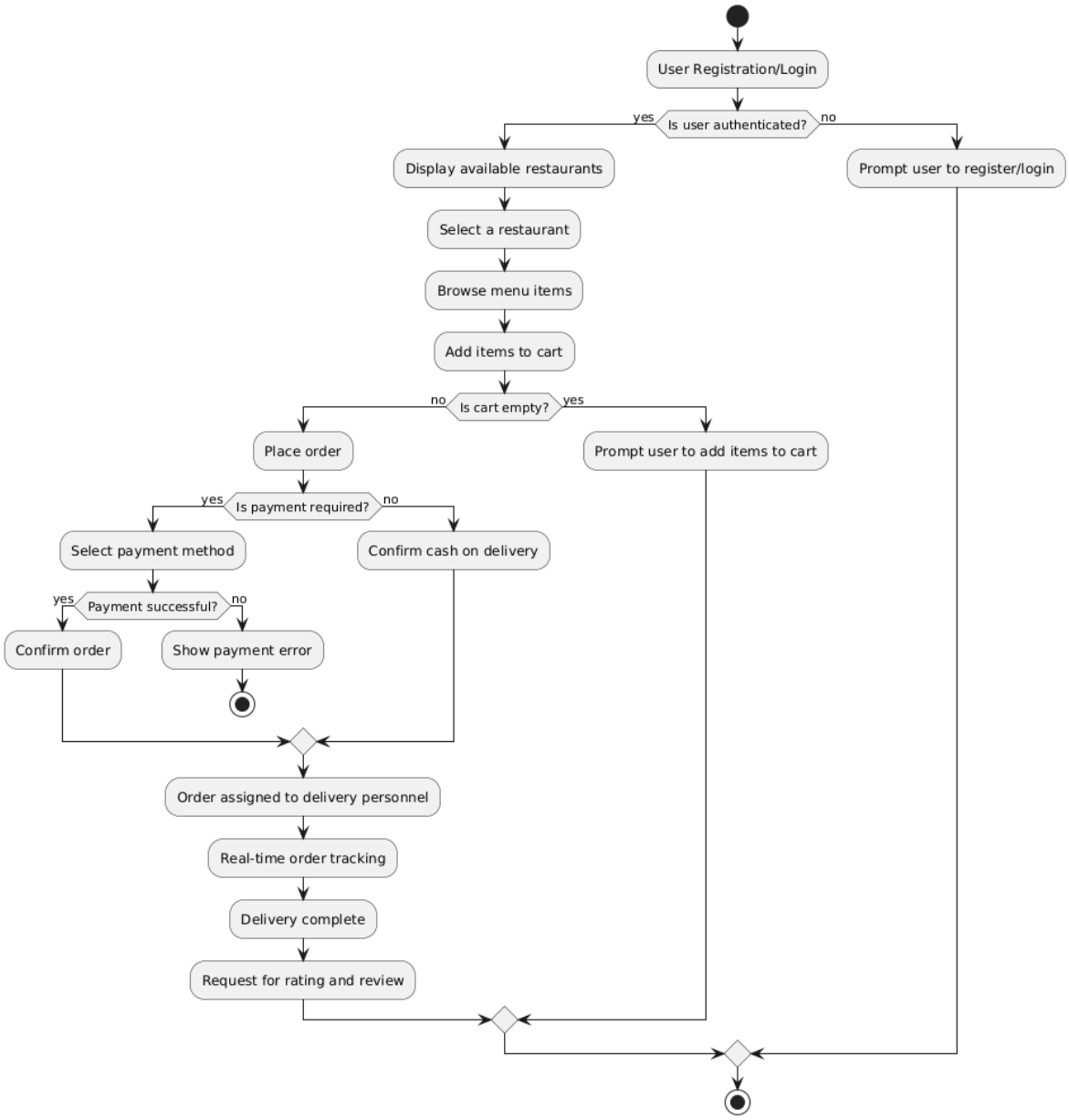

Figure 5.11: Flowchart

Chapter 6
Testing

Testing phase (as part of the Software Development Life Cycle for the software application) is one of the most important phases that guarantee the quality, performance, and efficiency of the software application. The testing process refers to executing the software/system in order to identify any flaws or discrepancies within its functioning. Testing plays an essential role in ensuring that the software conforms to the required requirements. Different kinds of testing techniques such as unit, integration, system, and acceptance testing are used for evaluating both units and the entire system. In summary, effective testing is not only about the timely identification of errors and mistakes but also concerns customer satisfaction in that the customer will have a product which works properly. With the current technological advances, testing is becoming increasingly important and a vital aspect of the software engineering process.[**?**].

6.1 Unit testing

Each module is tested individually to verify functionality.

- **Objective:** To validate that the Online Food Ordering and Delivery System meets all requirements and performs as expected.
- **Scope:** Covers the full system workflow, from user login to order completion and feedback.

6.1.1 User Authentication

- **Objective:** Ensure secure login, logout, and registion.

Table 6.1: User Authentication

No.	**Test Case**	**Input**	**Expected Result**	**Result**
1	Register user with valid data	Username, Password, Email	User account created and confirmation shown	Pass
2	Register user with missing fields	Username only	Error message for required fields	Pass
3	Login with correct credentials	Correct Username/Password	Redirect to user dashboard	Pass
4	Login with incorrect credentials	Incorrect Username/Password	Error message for invalid credentials	Pass
5	Logout user	Click on logout button	User is logged out and session terminated	Pass

6.1.2 Restaurant Management

- **Objective:** Allow restaurant registration, profile management.

Table 6.2: Restaurant Management

No.	Test Case	Input	Expected Result	Result
1	Register new restaurant	Restaurant details	Restaurant account created	Pass
2	Edit restaurant profile	New contact info	Profile updated successfully	Pass
3	Add menu item	Item Name, Price, Description	Item is added to the menu	Pass
4	Edit menu item details	Updated item details	Item details updated in the menu	Pass
5	Delete menu item	Select menu item	Item removed from the menu	Pass

6.1.3 Ordering System

- **Objective:** Enable browsing, viewing, and ordering from restaurant menus.

Table 6.3: Ordering System

No.	Test Case	Input	Expected Result	Result
1	Browse restaurants	Select category or location filter	List of relevant restaurants is displayed	Pass
2	View restaurant menu	Click on restaurant	Full menu displays with item details	Pass
3	Add item to cart	Select item from menu	Item added to cart with correct quantity	Pass
5	Place order	Confirm cart and delivery details	Order is placed and order ID generated	Pass
6	Check order status	Enter order ID	Real-time order status displayed	Pass

6.1.4 Payment Processing

- **Objective:** Provide secure and functional payment options.

Table 6.4: Payment Processing

No.	Test Case	Input	Expected Result	Result
1	Select payment method	Choose Stripe	Redirects to payment gateway	Pass
2	Enter valid payment details	Card info, billing info	Payment successful and confirmation shown	Pass
3	Enter invalid payment details	Expired card info	Payment fails and error message shown	Pass
4	Cash on delivery option	Select COD	Confirmation shown and no online payment needed	Pass

6.1.5 Delivery Management

- **Objective:** Manage delivery assignments and track orders.

Table 6.5: Delivery Management

No.	Test Case	Input	Expected Result	Result
1	Assign delivery personnel to order	Order ID, Delivery personnel ID	Delivery personnel assigned to the order	Pass
2	Update delivery status	Delivery ID, Status	Status updated in real-time	Pass
3	Track delivery in real time	Delivery ID	Map displays delivery personnel's live location	Pass

6.1.6 Chatbot for Customer Support

- **Objective:** Provide a simple, functional chatbot for customer support.

Table 6.6: Chatbot Testing

No.	Test Case	Input	Expected Result	Result
1	Ask a common question	"How do I track my order?"	Chatbot provides answer from FAQ database	Pass
2	Ask an uncommon question	"How do I contact support?"	Chatbot redirects to customer support contact	Pass
3	Escalation to human agent	Unanswered question	Chatbot offers escalation to live chat or email	Pass

6.1.7 Ratings and Reviews

- **Objective:** Collect and display customer ratings and reviews.

Table 6.7: Ratings and Reviews

No.	Test Case	Input	Expected Result	Result
1	Submit rating after order completion	Rating (1-5), Review text	Rating and review saved and shown in profile	Pass
2	Edit review	Updated rating or review text	Updated feedback shown in profile	Pass
3	View average rating for restaurant	Access restaurant profile	Average rating displayed based on all reviews	Pass

6.2 Functional Testing

Functional testing focuses on the performance of each module as specified in the requirements.

- **Objective:** It focuses on the features of the application and how well they function according to specifications.

6.2.1 Ordering system

- **Objective:** Verify the process from order creation to tracking.

Table 6.8: Ordering System

No.	**Test Case**	**Scenario**	**Expected Result**	**Result**
1	Browse restaurant menus	Select a restaurant	Full menu displays with item descriptions and prices	Pass
2	Add items to cart	Choose item, specify quantity	Item appears in cart with correct details	Pass
3	View cart and modify items	Access cart, change quantity	Cart updates with correct item quantities and prices	Pass
4	Place order	Confirm items and enter address	Order confirmation and ID are provided	Pass
5	Track order status	Check current order	Real-time status updates visible	Pass

6.2.2 Payment Processing

- **Objective:** Ensure the payment processing system functions correctly for various payment methods.

Table 6.9: Payment Processing

No.	Test Case	Payment Method	Expected Result	Result
1	Process payment via Stripe	Credit Card	Payment is successful with confirmation shown	Pass
2	Attempt payment with invalid card details	Expired Card	Payment fails with error message	Pass
3	Select cash on delivery	Cash	Order placed without requiring online payment	Pass

6.2.3 Delivery Management

- **Objective:** Ensure delivery assignment and tracking work as required.

Table 6.10: Delivery Management

No.	Test Case	Scenario	Expected Result	Result
1	Assign delivery personnel to order	Assign delivery ID to order	Delivery personnel assigned successfully	Pass
2	Update delivery status	Delivery status updated	Customer sees updated status in real-time	Pass
3	Confirm delivery completion	Delivery marked as completed	Delivery marked as completed successfully	Pass

6.2.4 Customer Support and Chatbot

- **Objective:** Verify that the support features are accessible and functional.

Table 6.11: Functional Testing for Customer Support and Chatbot

No.	Test Case	Scenario	Expected Result	Result
1	Access FAQ section	Click FAQ	FAQ page with common questions shown	Pass
2	Start live chat with rider	Select live chat option	Chat interface opens	Pass
3	Ask a question to the chatbot	Ask a common question	Chatbot provides correct answer	Pass

6.3 Integration Testing

Integration testing ensures that modules interact correctly with each other to provide a seamless user experience.

6.3.1 Order and Payment Integration

- **Objective:** Confirm the seamless integration between ordering and payment modules.

Table 6.12: Order and Payment Integration

No.	Test Case	Scenario	Expected Result	Result
1	Place order and proceed to payment	Order confirmed, redirect to payment	Payment gateway displays correct order amount	Pass
2	Payment confirmation	Complete payment	Payment confirmed and order marked as ”Paid”	Pass

6.3.2 Order Placement to Delivery Tracking

- **Objective:** Verify smooth workflow from placing an order to tracking delivery.

Table 6.13: Order Placement to Delivery Tracking Integration

No.	Test Case	Scenario	Expected Result	Result
1	Place order and assign to delivery personnel	Order placed successfully	Delivery personnel assigned to order	Pass
2	Track assigned order	Order ID provided	Real-time delivery tracking visible to customer	Pass
3	Confirm order delivered	Delivery marked complete	Order status updates to ”Delivered” for customer	

6.4 Automated Testing

Automated testing scripts for repetitive tasks, focusing on UI responsiveness, form validation, and order tracking.

6.4.1 Responsive Design Testing

- **Objective:** Ensure the platform displays correctly on various device types.

Tools Used: Selenium or BrowserStack for device emulation.

Table 6.14: Responsive Design Testing

Device	**Expected Layout**	**Result**
Smartphone	Optimized for small screens, easy navigation	Pass
Tablet	Slightly larger layout for readability	Pass
Desktop	Full-featured layout with side navigation	Pass

6.4.2 Form Validation Testing

- **Objective:** Validate all required fields and enforce correct input formats.

Table 6.15: Form Validation Testing

No.	**Test Case**	**Input**	**Expected Result**	**Result**
1	Login with valid credentials	Correct Username/Password	Redirects to user dashboard	Pass
2	Register with invalid email format	Missing ”@” in email	Error message for invalid email format	Pass
3	Payment with valid card	Correct card details	Payment processes without issues	Pass

6.5 System Testing

System testing validates the entire system's functionality by verifying end-to-end workflows and ensuring that all modules interact as expected.

Table 6.16: System Testing

No.	Test Scenario	Expected Result	Result
1	Register, login, and browse restaurants	User can register, log in, and view available restaurants without errors.	Pass
2	Place an order and track its status	Order is placed successfully and status updates (e.g., "In Progress," "Out for Delivery").	Pass
3	Use multiple payment methods (Stripe, Cash on Delivery)	Each payment method processes successfully with secure data handling.	Pass
4	Assign delivery personnel to an order and update delivery status	Delivery personnel assignment works and real-time tracking updates accurately.	Pass
5	Submit and view restaurant ratings and reviews	Ratings and reviews save correctly and display under the restaurant's profile.	Pass
6	Access customer support and use chatbot for help	Customer support channels and chatbot are functional and responsive.	Pass
7	Responsive design on different devices (mobile, tablet, desktop)	User interface adjusts appropriately across devices for usability.	Pass

Chapter 7
Conclusion and Future Work

Conclusion

In this project, we have successfully developed a comprehensive platform phase that ease restaurant direction, order processing, and client feedback through ratings and review. The system has undergone rigorous testing to ensure functionality, security, and us ability across various substance abuser scenario. Overall, the chopine meets the project objective and allow for a user - well-disposed experience for both customer and restaurant owners.

Future Work

- **Enhanced User Experience:** Adding more advanced features such as personalized recommendations that recommend based on user preferences and order history.
- **Mobile Application Development:** Creating a dedicated mobile application to enhance usability and suitability for those who are always on the move.
- **Integration of AI Chatbots:**Implementing AI driven chatbots by integrating them on the website for help assist customers in sorting out problems immediately.
- **Data Analytics:** Integrating analytic tools in the system to collect data from user interactions and modify services to fit the clients' needs as they change.
- **Expansion of Payment Options:** Incorporating more payment gateways to allow users the flexibility and ease of making purchases.

Bibliography

[1] M. Ullah, H. Qiu, W. Huangfu, D. Yang, Y. Wei, and B. Tang, "Integrated machine learning approaches for landslide susceptibility mapping along the pakistan–china karakoram highway," *Land*, vol. 14, no. 1, 2025. [Online]. Available: https://www.mdpi.com/2073-445X/14/1/172

[2] A. Rehman, J. Song, F. Haq, S. Mahmood, M. I. Ahamad, M. Basharat, M. Sajid, and M. S. Mehmood, "Multi-hazard susceptibility assessment using the analytical hierarchy process and frequency ratio techniques in the northwest himalayas, pakistan," *Remote Sensing*, vol. 14, no. 3, 2022. [Online]. Available: https://www.mdpi.com/2072-4292/14/3/554

Appendices

Appendix A - Turnitin Similarity Report

This appendix contains the official Turnitin Similarity Report generated for the final submitted version of this dissertation.

Plagiarism Report Screenshot:

Appendix B - AI Writing Detection Report

This appendix contains the AI Writing Detection Report generated by Turnitin (if applicable), verifying compliance with institutional academic integrity guidelines regarding AI-assisted content.

AI Detection Report Screenshot:

Appendix C – Source Code

This section provides details about the project source code repository.

Guidelines for Students:

- Upload your complete project source code to a version control platform such as GitHub, GitLab, or Bitbucket.
- Ensure the repository includes:
  - All source files
  - Database scripts (if applicable)
  - Model training and testing code (for ML projects)
  - Frontend and backend code (for web/mobile projects)
  - README file with setup instructions
- The repository must be properly structured with meaningful folder names.
- Remove unnecessary files (temporary files, cache files, datasets if restricted).
- If the repository is private, make the repository public before final submission.
- Paste the final repository link below.

Repository Link:

`PasteYourGitHubRepositoryLinkHere`

Additional Notes:

- Mention branch name if different from `main`.
- Mention if any third-party APIs require API keys.
- Mention hardware requirements (if ML/GPU-based project).